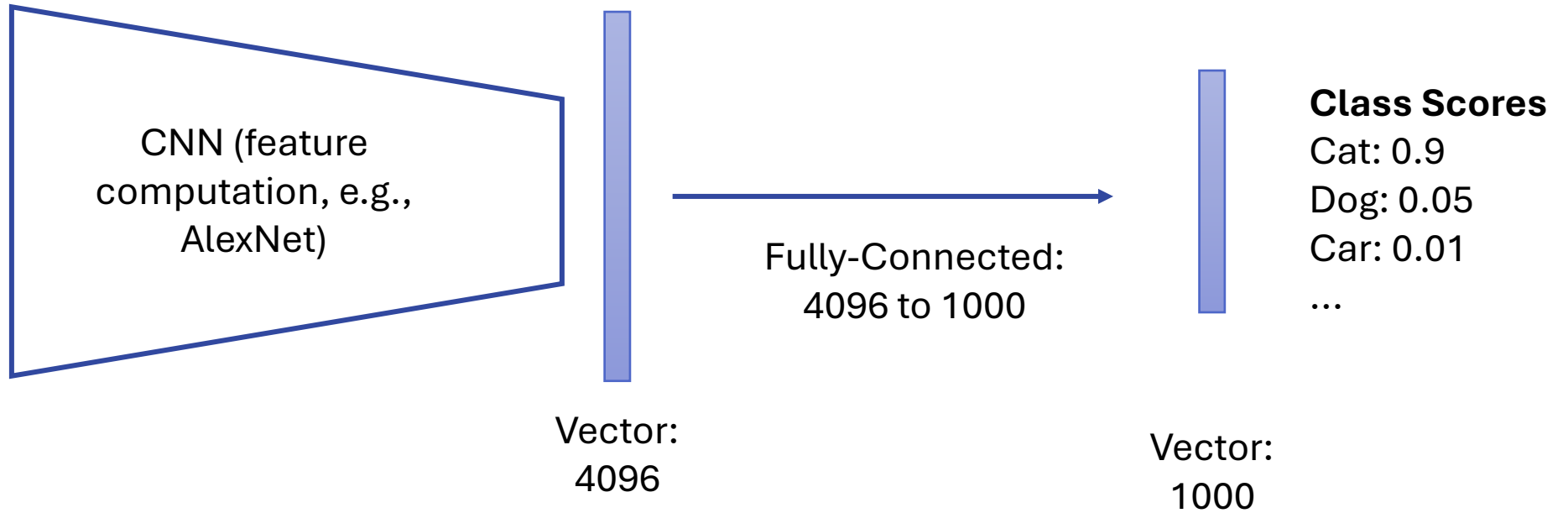


Object Detection

Slides have been adapted from Justin Johnson.

So Far: Image Classification



Vision Tasks

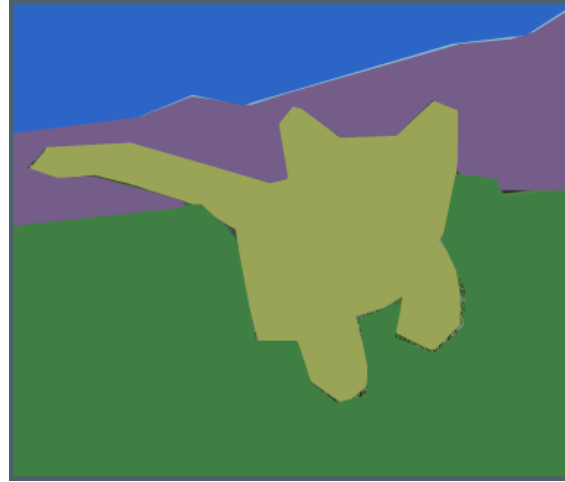
Classification



CAT

No spatial extent

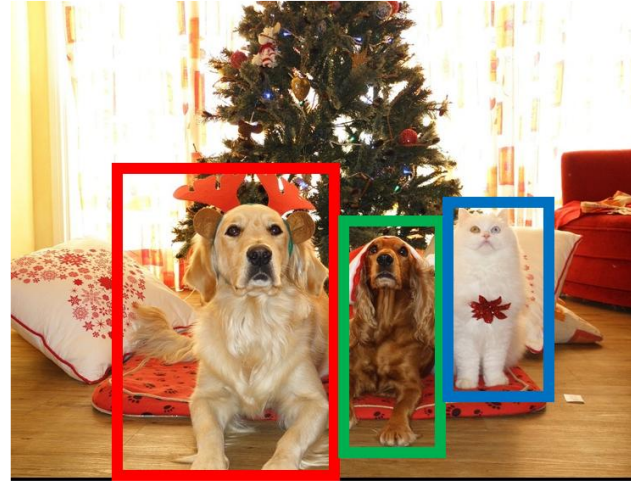
Semantic Segmentation



GRASS, CAT,
TREE, SKY

No object, just
pixels

Object Detection



DOG, DOG, CAT

Multiple objects

Instance Segmentation



DOG, DOG, CAT

Object Detection: Task Definition

Input:

Single RGB Image

Output:

A set of detected objects;

For each object predict:

1. **Category label** (from fixed, known set of categories)
2. **Bounding box** (four numbers: x, y, width, height)

How to detect bounding box?
Data set must have ROF or ground truth for training



Object Detection: Challenges

- **Multiple outputs:** Need to output variable numbers of objects per image
- **Multiple types of output:** Need to predict “what” (category label) as well as “where” (bounding box) *— classify*
- **Large images:** Classification works at 224x224; need higher resolution for detection, often ~800x600 *— box*

*Total multiclass
loss = classification loss + regression loss*

*classification → what
detection → where*



Detecting a Single Object



Often pretrained on
ImageNet (Transfer
learning)

CNN (feature
computation, e.g.,
AlexNet)

Vector:
4096

Fully-Connected:
4096 to 1000

Vector:
1000

Fully-Connected:
4096 to 4

What

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

**Correct
Label**
Cat

**Softmax
Loss**

**Multitask
Loss**

**Weighted
Sum** → **Loss**

**Box
Coordinates**
(x, y, w, h)

L2 Loss

Correct Box
(x', y', w', h')

Where

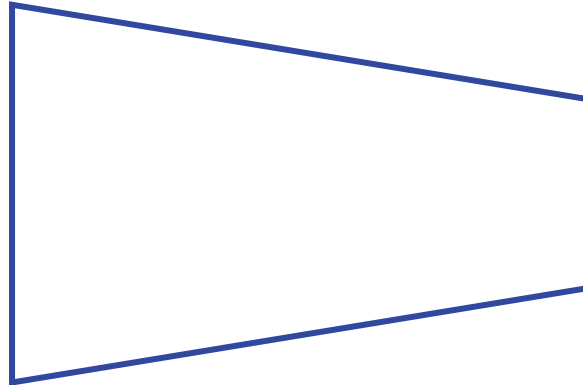
**Treats localization as a
regression problem.**

**Problem: Images can
have more than one
object.**

Treat localization as a regression problem.

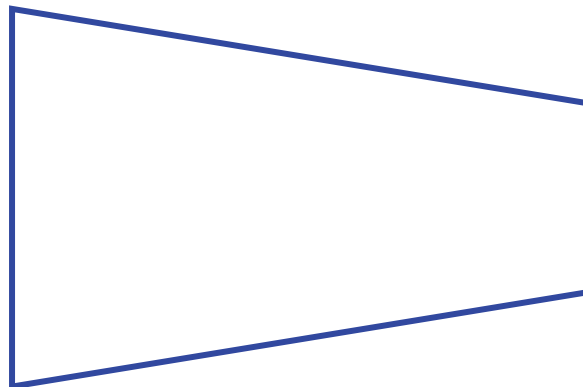
Detecting a Multiple Objects

Need different numbers of
outputs per image



CAT: (x, y, w, h)

4 numbers

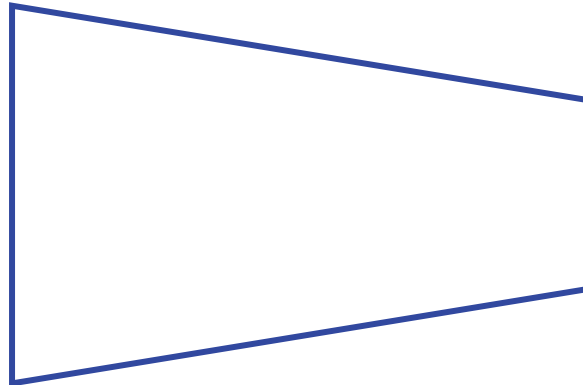


DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)

12 numbers



DUCK: (x, y, w, h)

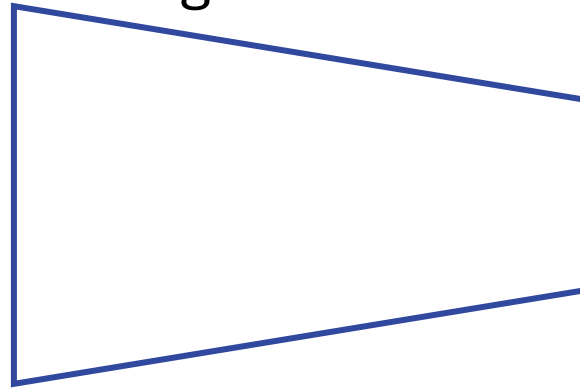
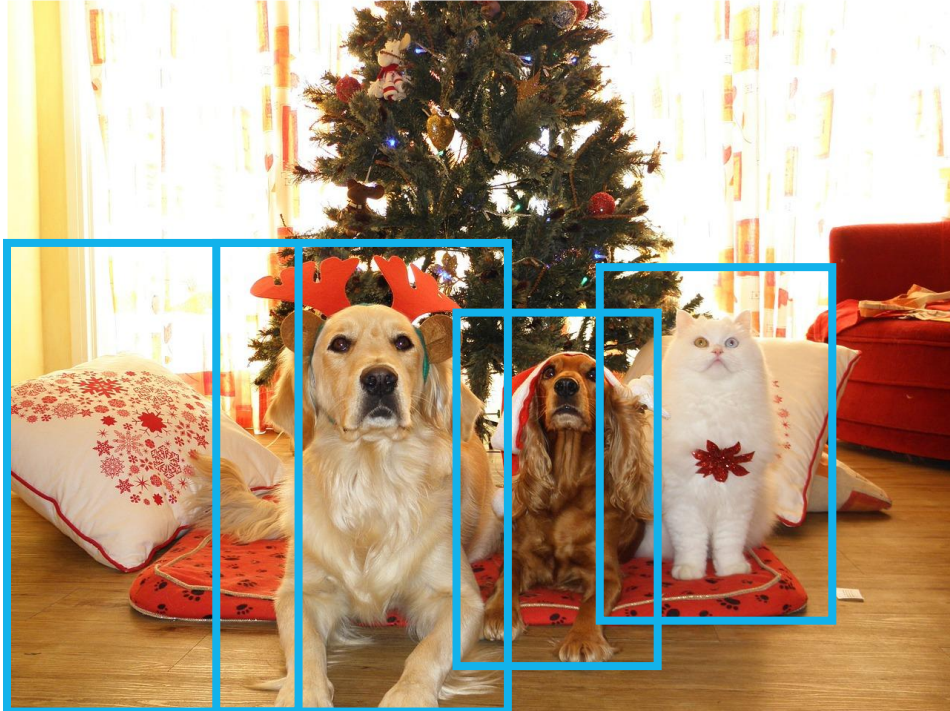
DUCK: (x, y, w, h)

...

Many numbers

Detecting a Multiple Objects: Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

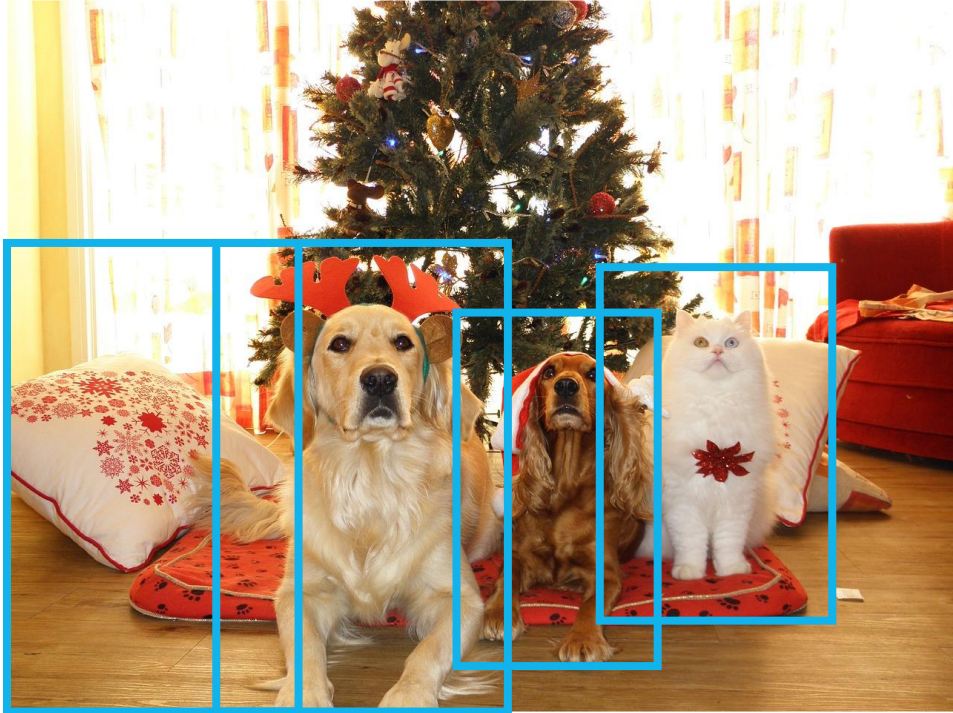


DOG? ~~YES~~

CAT? ~~YES~~

Background? ~~YES~~

Detecting a Multiple Objects: Sliding Window



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

Question: How many possible boxes are there in an image of size $H \times W$?

Consider a box of size $h \times w$:

Possible x positions: $W - w + 1$

Possible y positions: $H - h + 1$

Possible positions:

$(W - w + 1) \times (H - h + 1)$

800 x 600 image
has ~58M boxes!
No way we can
evaluate them all.

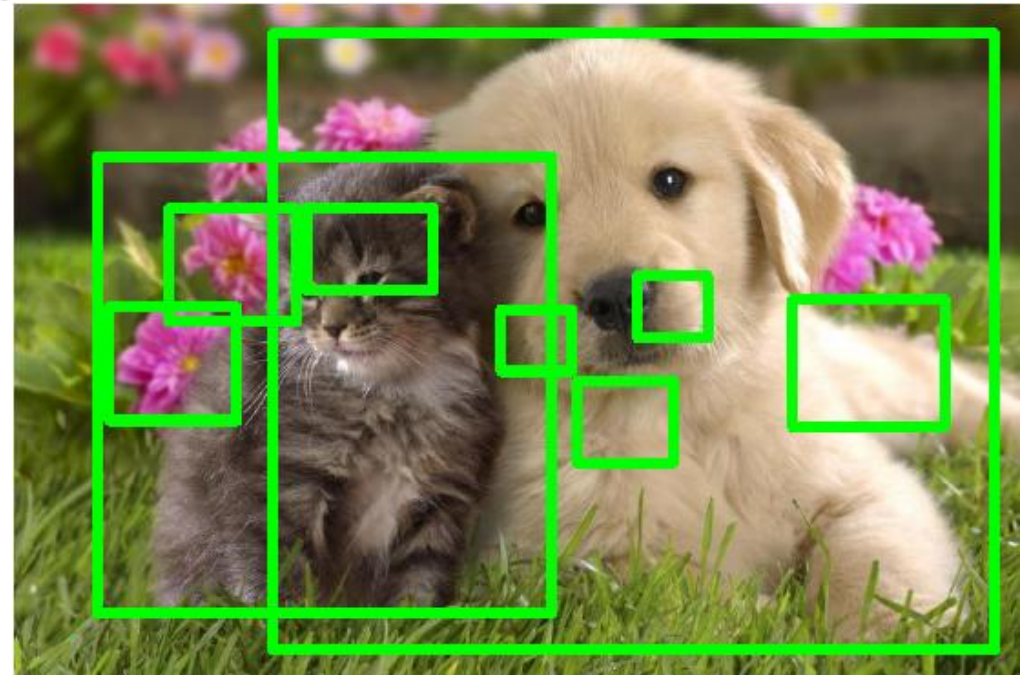
Total possible boxes:

$$\sum_{h=1}^H \sum_{w=1}^W (W - w + 1) \times (H - h + 1) = \frac{H(H+1)}{2} \frac{W(W+1)}{2}$$

Region Proposals

- Find a small set of boxes that are likely to cover all objects
- Often based on heuristics: e.g. look for “blob-like” image regions
- Relatively fast to run; e.g., Selective Search gives 2000 region proposals in a few seconds on CPU

Now only run CNN 2000 times



Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012

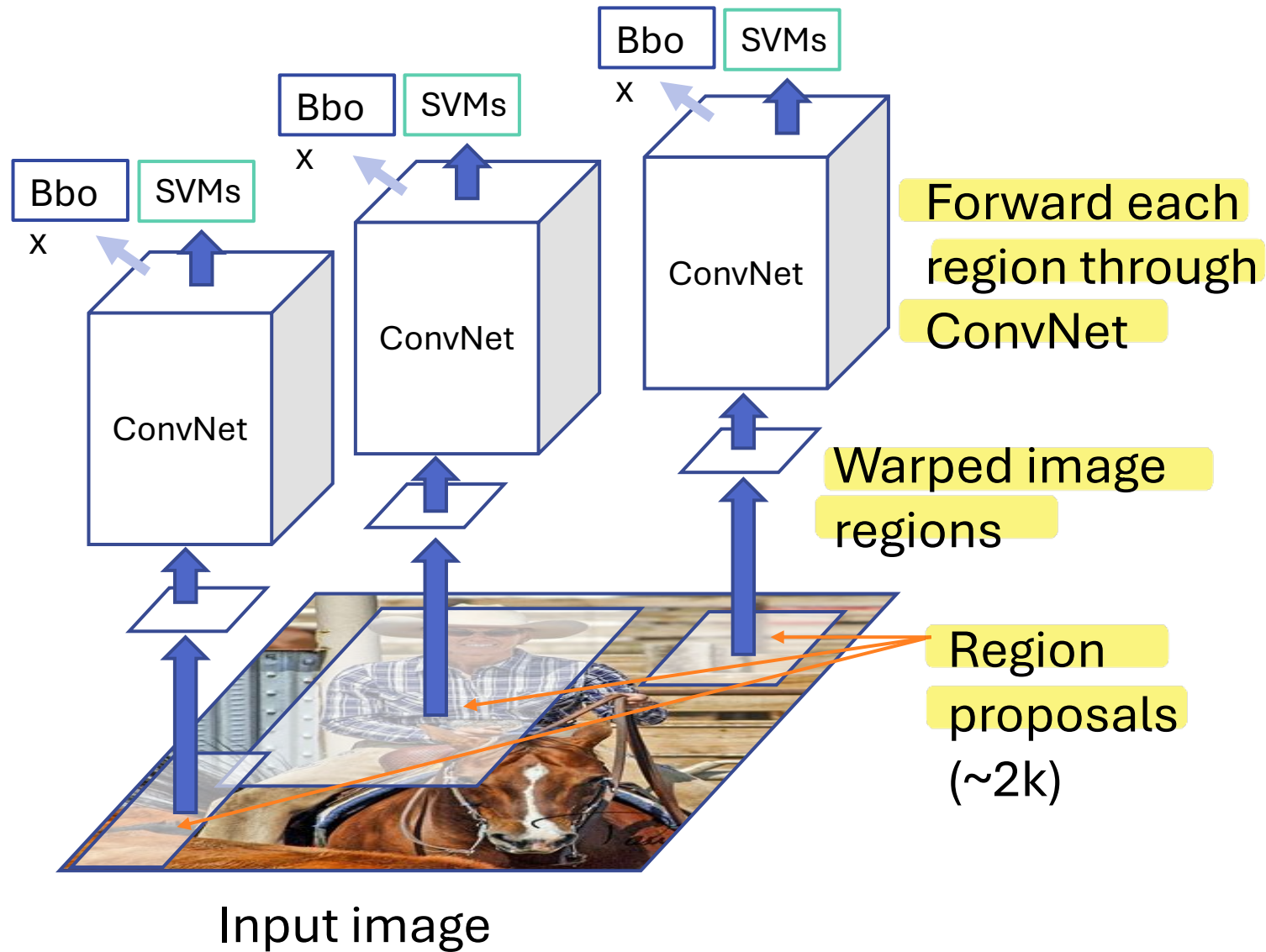
Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013

Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014

Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

R-CNN: Region Proposals + CNN Features

Source: R. Girshick



Classify regions with SVMs

Bounding box regression:
Predict “transform” to correct the
RoI: 4 numbers (t_x, t_y, t_h, t_w)

Region proposal: (p_x, p_y, p_h, p_w)

Transform: (t_x, t_y, t_h, t_w)

Output box: (b_x, b_y, b_h, b_w)

Translate relative to box size:

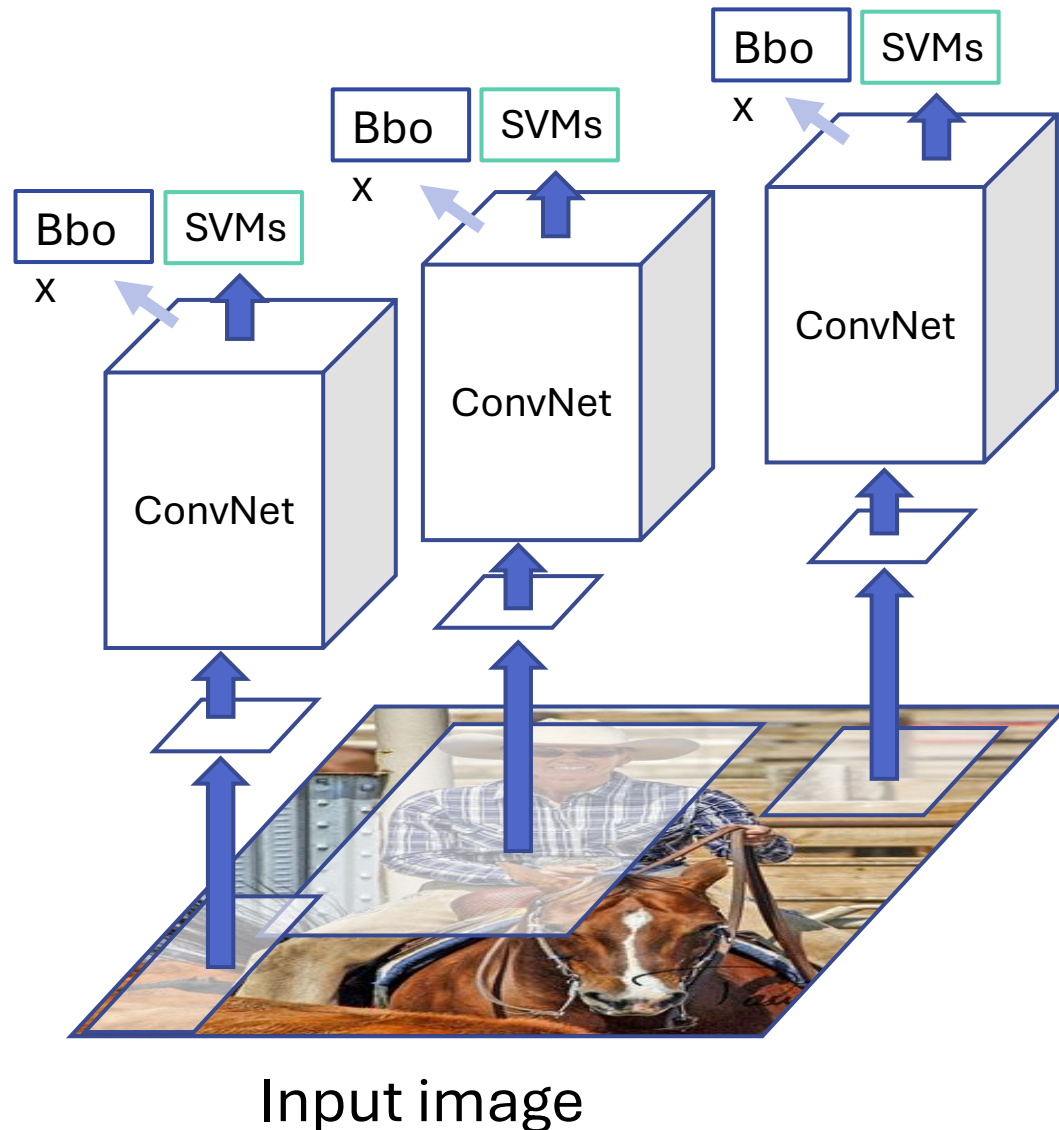
$$b_x = p_x + p_w t_x, b_y = p_y + p_h t_y$$

Log-space scale transform:

$$b_w = p_w \exp(t_w), b_h = p_h \exp(t_h)$$

R-CNN: Test Time

Source: R. Girshick



Input: Single RGB Image

1. Run region proposal method to compute ~2000 region proposals
2. Resize each region to **224×224** and run independently through CNN to predict class scores and bbox transform
3. Use scores to select a subset of region proposals to output (Many choices here: threshold on background, or per-category? Or take top K proposals per image?)
4. Compare with ground-truth boxes

Comparing Boxes: Intersection over Union (IoU)

How can we compare our prediction to the ground-truth box?

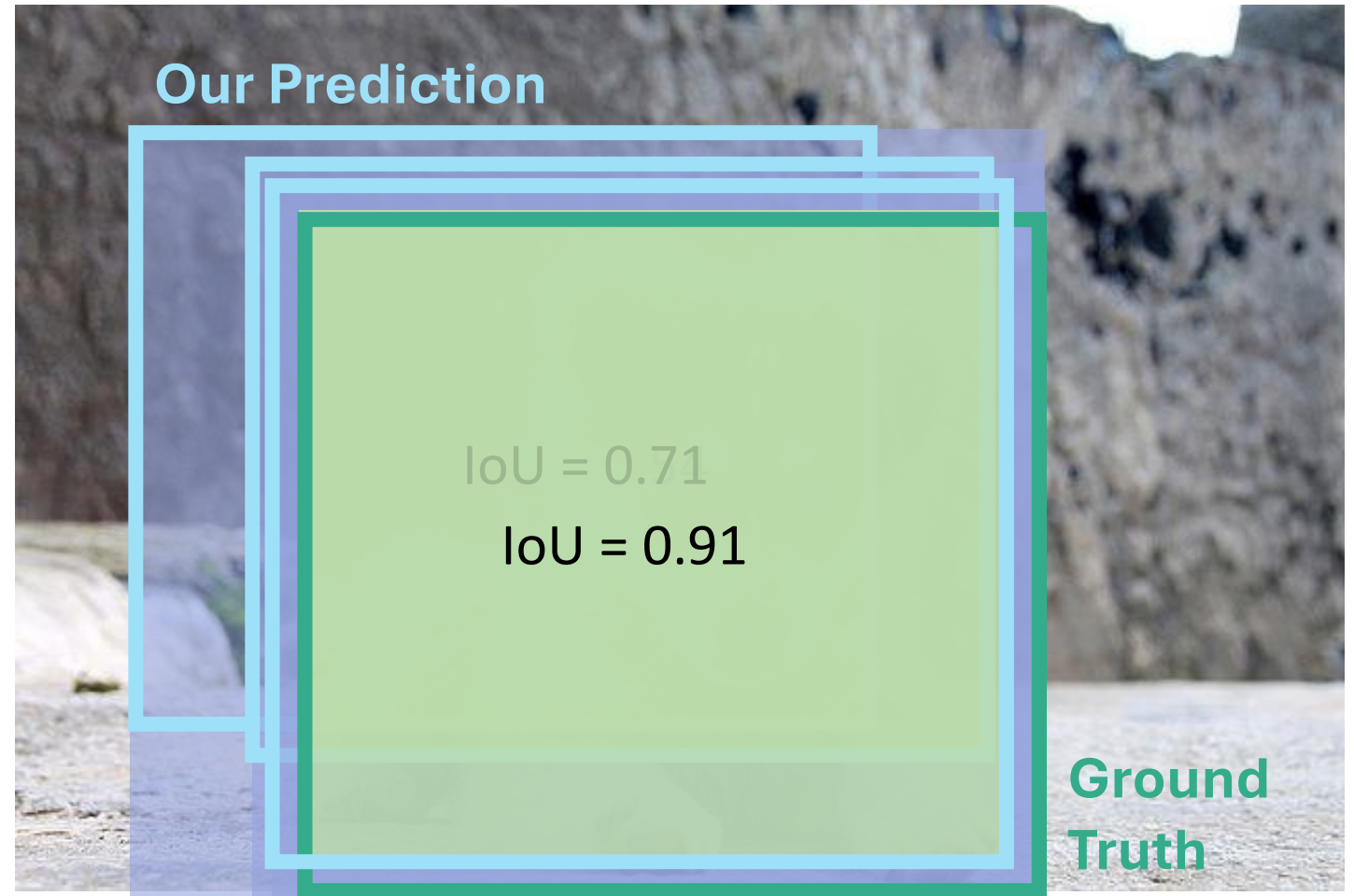
Intersection over Union (IoU)
(Also called “Jaccard similarity” or “Jaccard index”):

$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$

IoU > 0.5 is “decent”

IoU > 0.7 is “pretty good”

IoU > 0.9 is “almost perfect”



Evaluating Object Detectors

- **Intersection over Union (IoU)**

IoU is a metric that measures the overlap between two bounding boxes: the predicted box and the ground-truth box: 0 (no overlap) to 1 (perfect overlap).

- **Non-Maximum Suppression (NMS)**

After the detection step, NMS remove redundant bounding boxes. Among multiple detections of the same object, the box with the highest confidence score is kept, and others with an IoU greater than a set threshold are suppressed. This reduces duplicate detections.

- **Average Precision (AP)**

AP measures how well a detector ranks predictions. It is the area under the precision–recall curve (PR curve) for one class.

- **Mean Average Precision (mAP)**

mAP is the mean of AP values across all object classes. mAP provides an overall measure of detection performance for multi-class object detection.

- **COCO Protocol**

The COCO evaluation protocol (used in the COCO dataset benchmark) extends mAP with stricter criteria: **AP@[.5:.95]**: mean AP averaged over IoU thresholds from 0.5 to 0.95 in steps of 0.05. **AP50**: AP at IoU = 0.5, **AP75**: AP at IoU = 0.75 (stricter).

Evaluating Obj. Detectors: Mean Average Precision

1. Run object detector on all test images (with NMS) .
2. For each category, compute Average Precision (AP) = area under Precision vs Recall Curve
 1. For each detection (highest score to lowest score)
 2. Otherwise mark it as negative
 3. Plot a point on PR Curve
3. Mean Average Precision (mAP) = average of AP for each category

Car AP = 0.65

Cat AP = 0.80

Do AP = 0.86

mAP@0.5 = 0.77

Evaluating Obj. Detectors: Mean Average Precision

1. Run object detector on all test images (with NMS) .
2. For each category, compute Average Precision (AP) = area under Precision vs Recall Curve
 1. For each detection (highest score to lowest score)
 2. Otherwise mark it as negative
 3. Plot a point on PR Curve
3. Mean Average Precision (mAP) = average of AP for each category
4. For “COCO mAP”: Compute mAP@thresh for each IoU threshold (0.5, 0.55, 0.6, ..., 0.95) and take average

$$\text{mAP@0.5} = 0.77$$

$$\text{mAP@0.55} = 0.71$$

$$\text{mAP@0.60} = 0.65$$

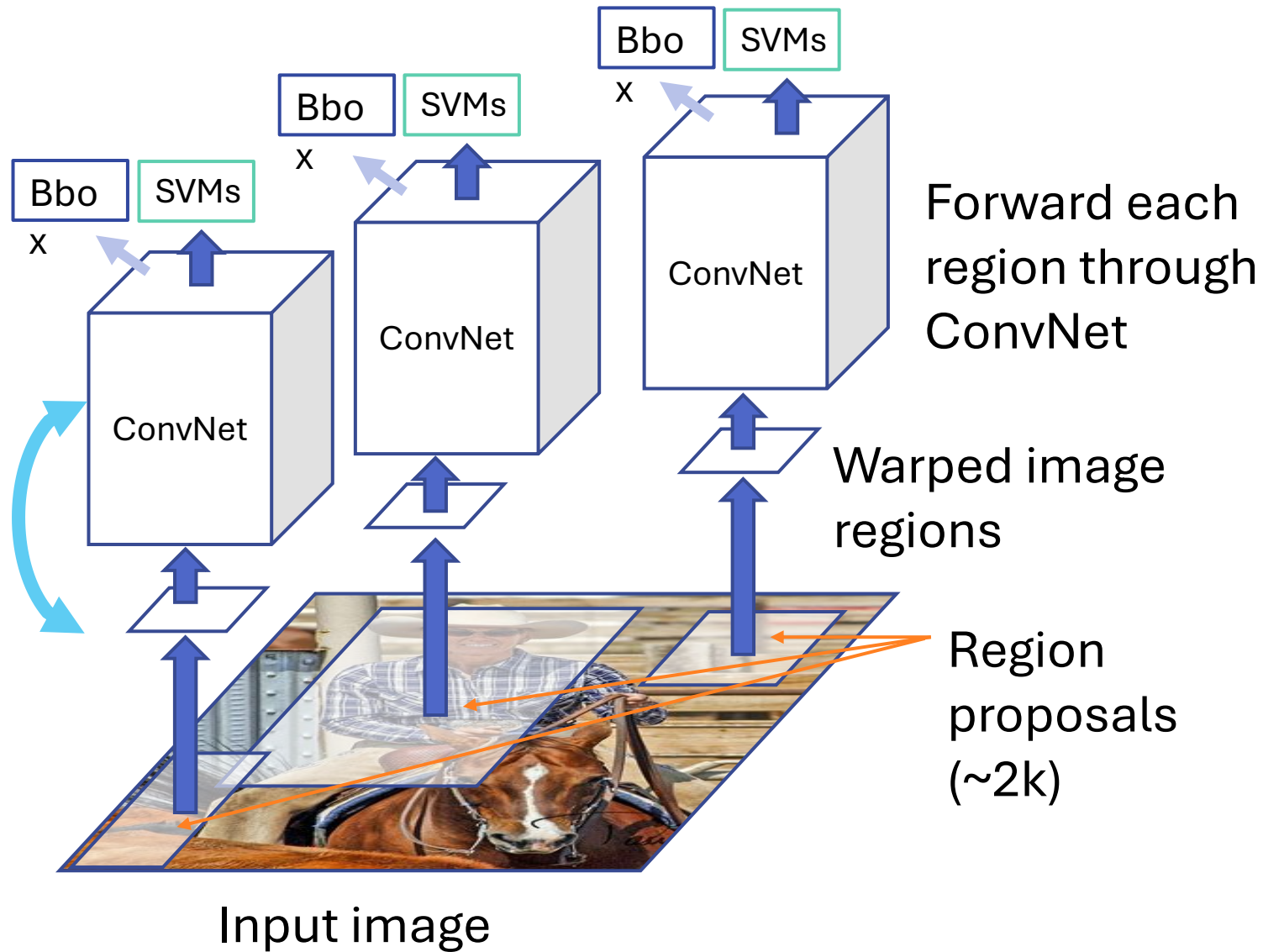
...

$$\text{mAP@0.95} = 0.2$$

$$\text{COCO mAP} = 0.4$$

R-CNN: Region-Based CNN

Source: R. Girshick



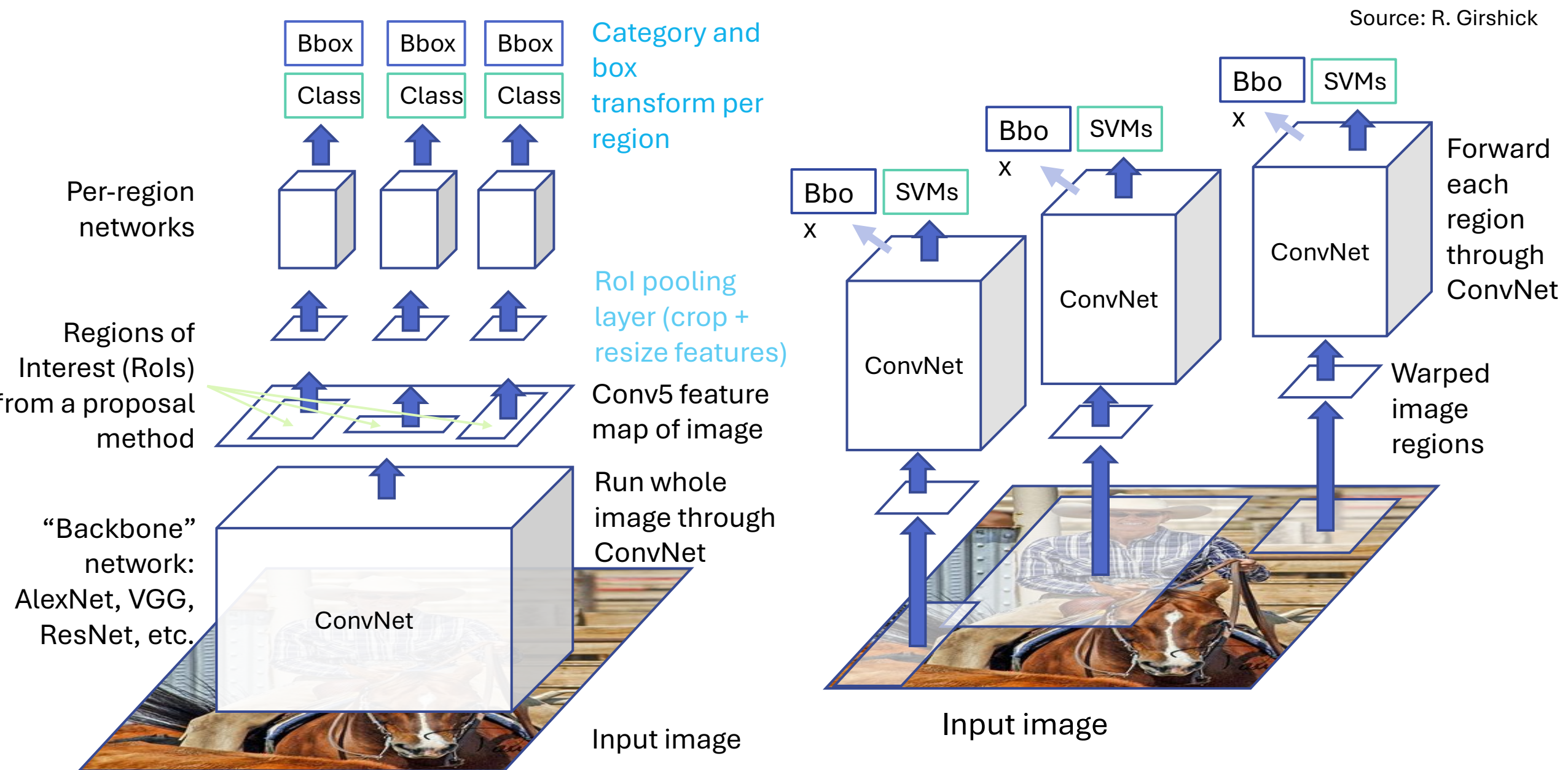
Classify regions with SVMs

Bounding box regression:
Predict “transform” to correct the
RoI: 4 numbers (t_x, t_y, t_h, t_w)

Problem: Very slow!
Need to do ~2k forward
passes for each image!

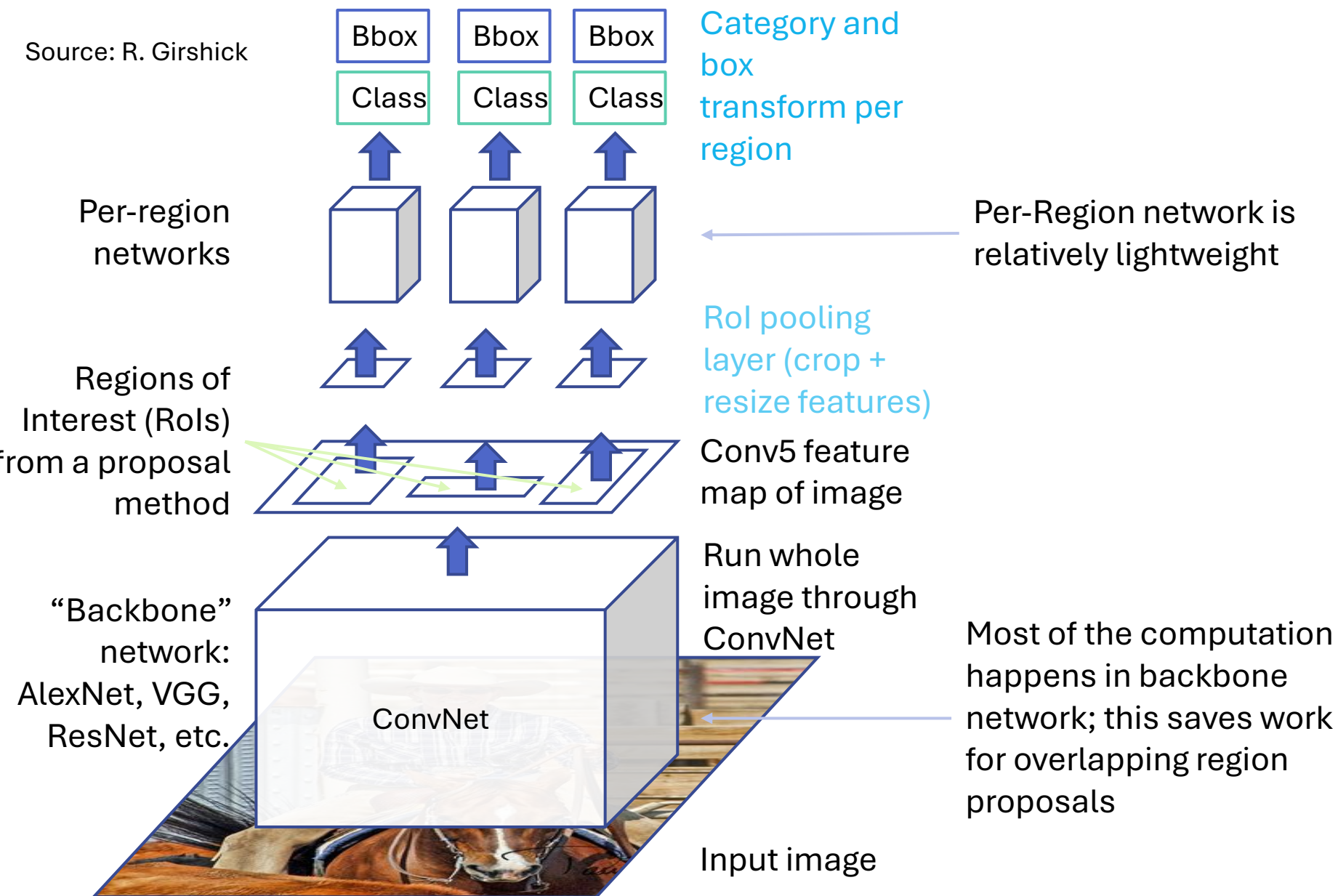
Solution: Run CNN
before warping!

Fast R-CNN



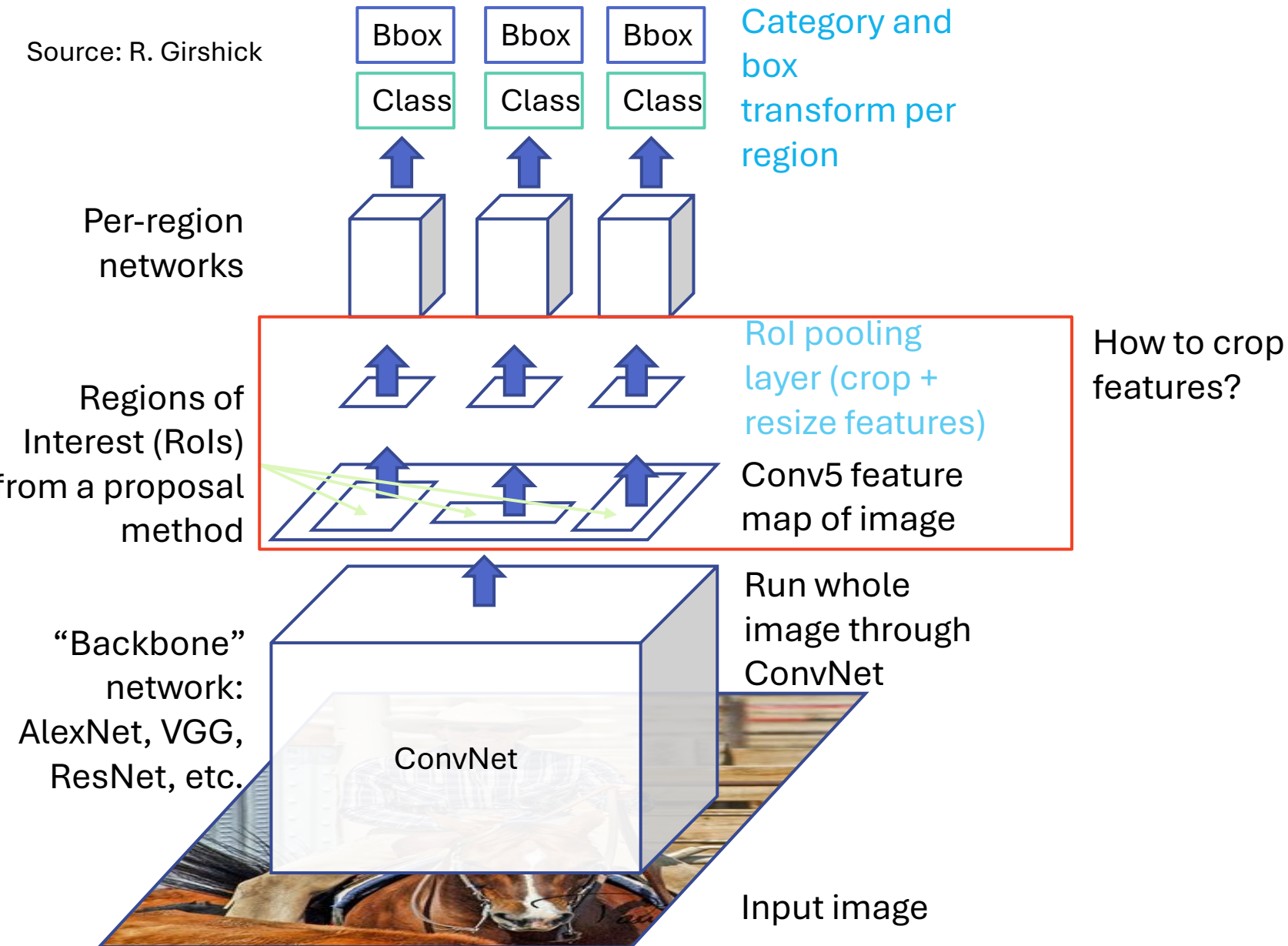
Fast R-CNN

Source: R. Girshick



Example: For PresNet last
stage list used detection region
networks are used for
between this and was 56 days
are used for per-region
network

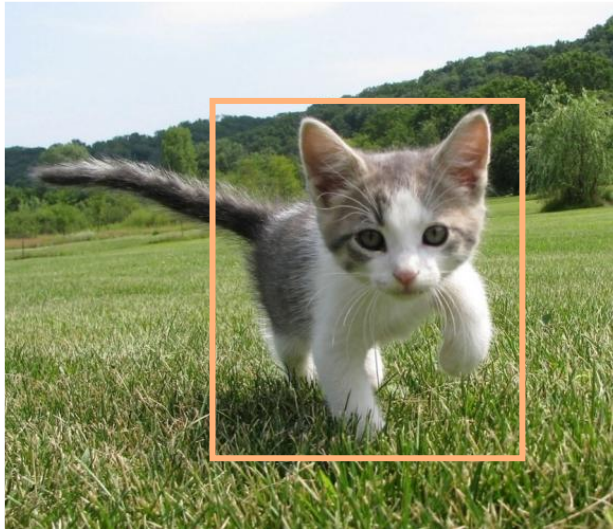
Fast R-CNN



Cropping Features: RoI Pool

Project object proposal
onto features

“Snap” to
grid cells



Input Image
(e.g., $3 \times 640 \times 480$)

CNN

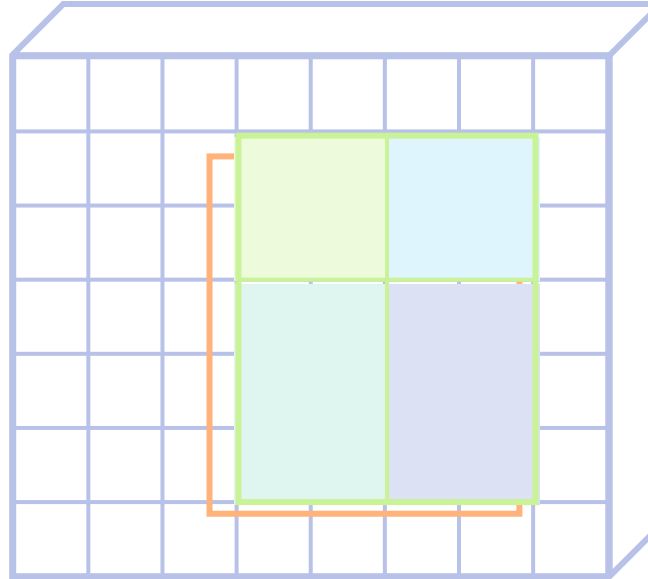
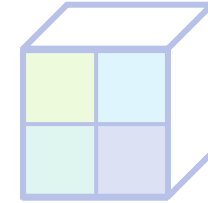


Image Features
(e.g., $512 \times 20 \times 15$)

Divide into 2x2
grid of (roughly)
equal subregions

Max-pool within
each subregion

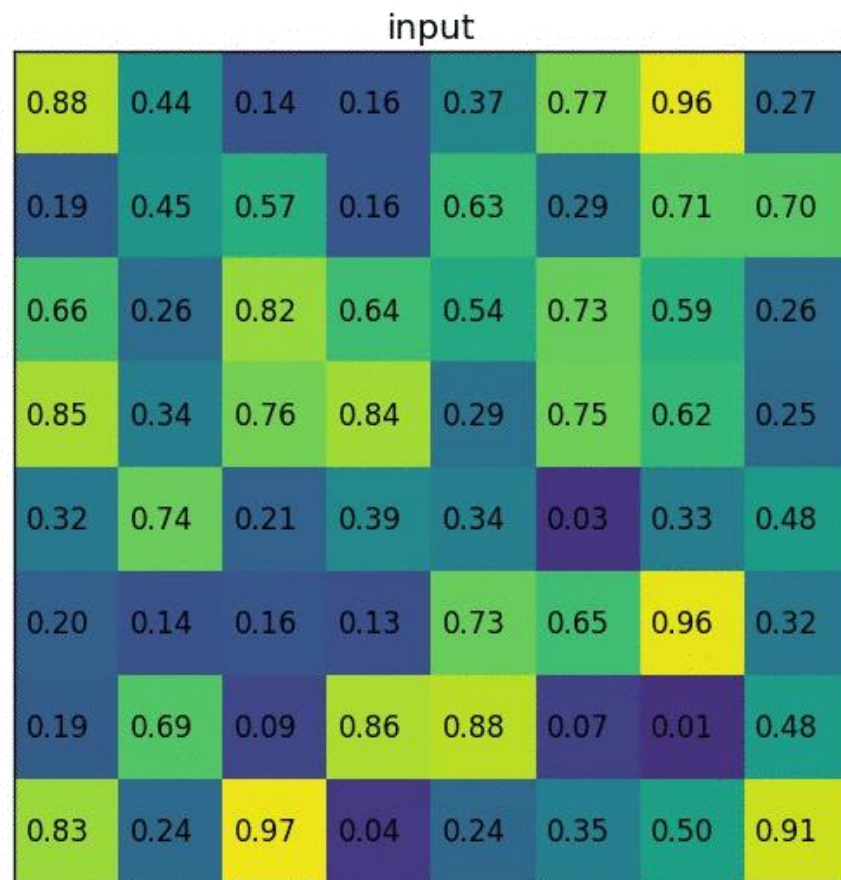


Region Features
(here $512 \times 2 \times 2$; in
practice, e.g., $512 \times 7 \times 7$)

Problem: Slight misalignment due to snapping; different-sized subregions is weird

Region features always the same size even if input regions have different sizes!

RoI Pooling Illustration



Fast R-CNN vs. “Slow” R-CNN

Fast R-CNN:

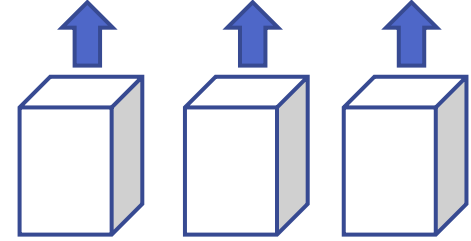
Apply differentiable cropping to shared image features

Per-region networks

Regions of Interest (Rols) from a proposal method

“Backbone” network: AlexNet, VGG, ResNet, etc.

Bbox
Class



Category and box transform per region

RoI pooling layer (crop + resize features)

Conv5 feature map of image

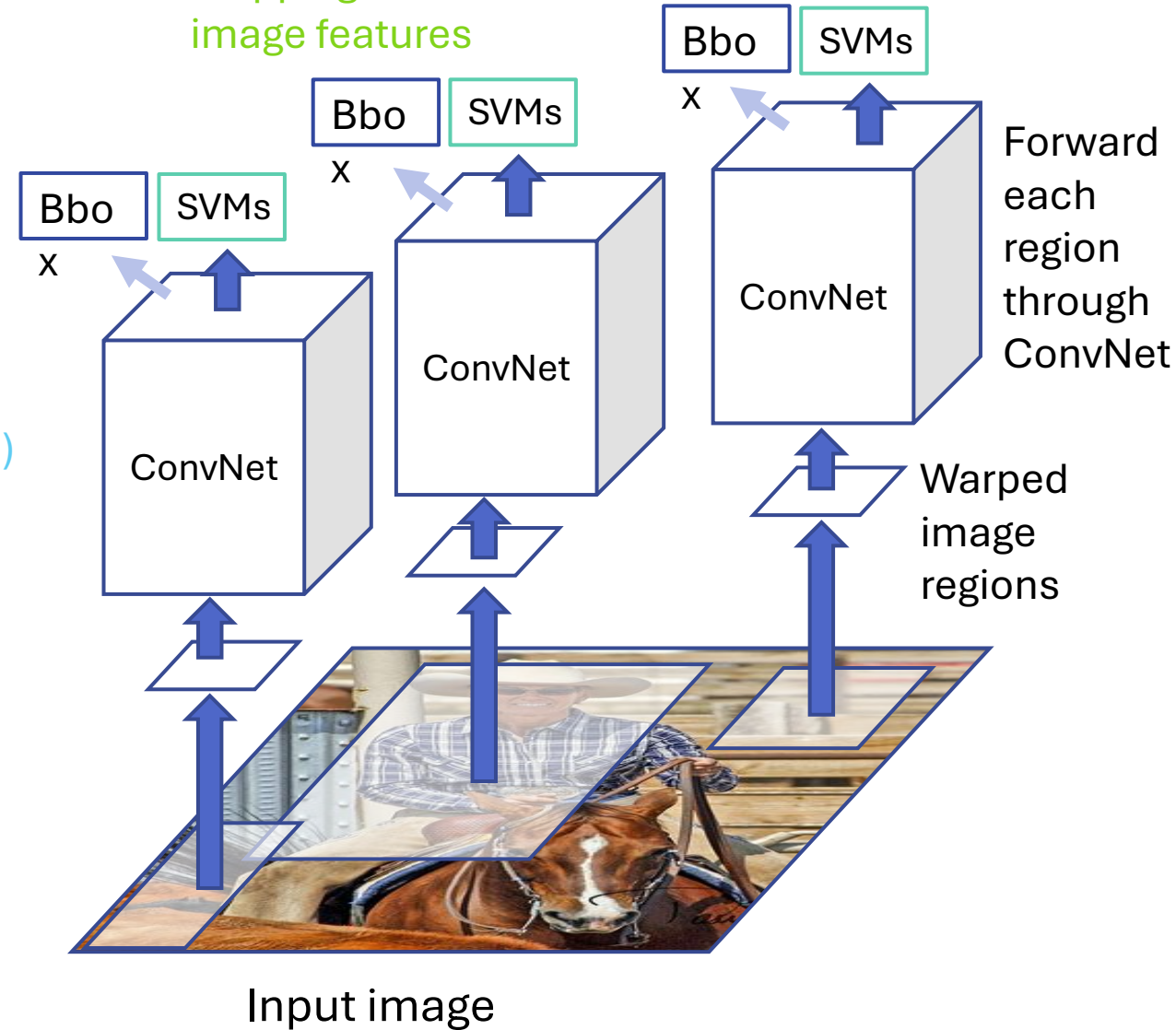
Run whole image through ConvNet

Input image

“Slow” R-CNN:

Apply differentiable cropping to shared image features

Source: R. Girshick



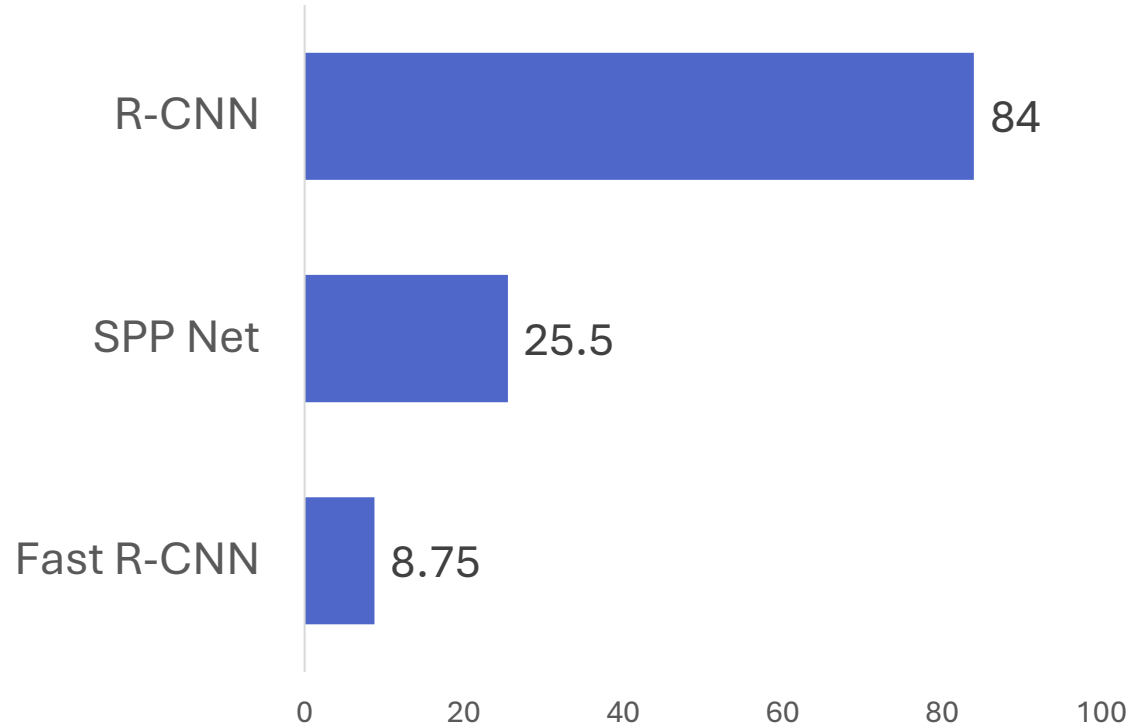
Forward each region through ConvNet

Warped image regions

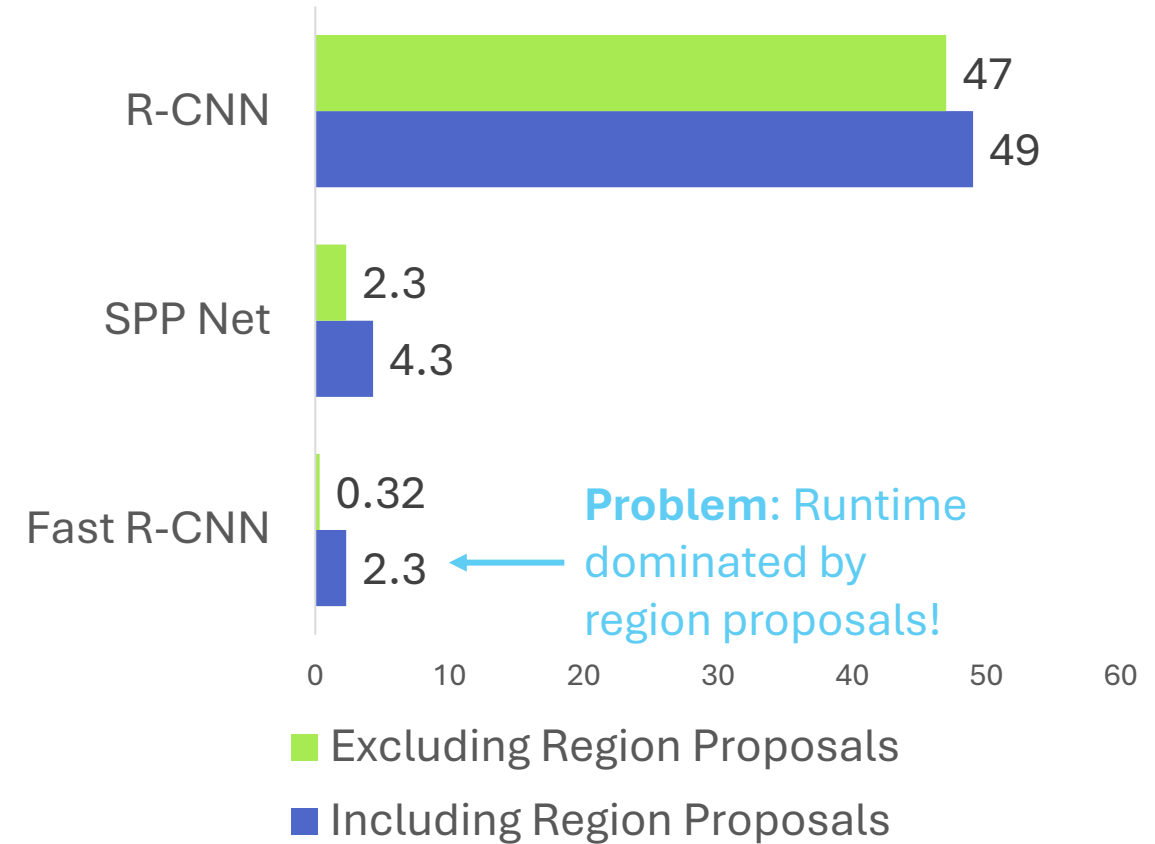
Input image

Fast R-CNN vs. “Slow” R-CNN

Training time (hours)



Testing time (seconds)



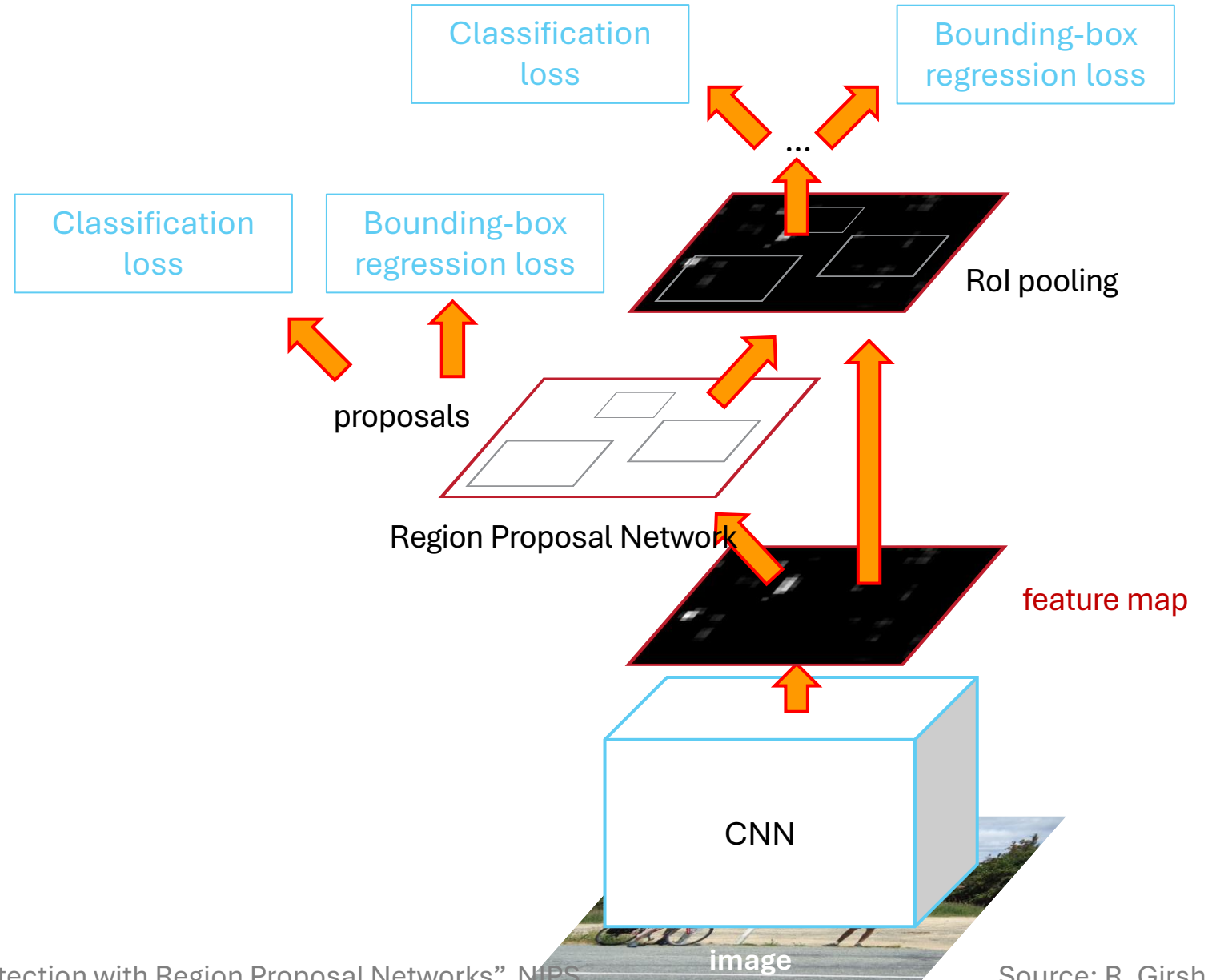
Problem: Runtime dominated by region proposals!

Recall: Region proposals computed by heuristic “Selective Search” algorithm on CPU -- let’s learn them with a CNN instead!

Faster R-CNN: Learnable Region Proposals

Insert Region Proposal Network (RPN) to predict proposals from features

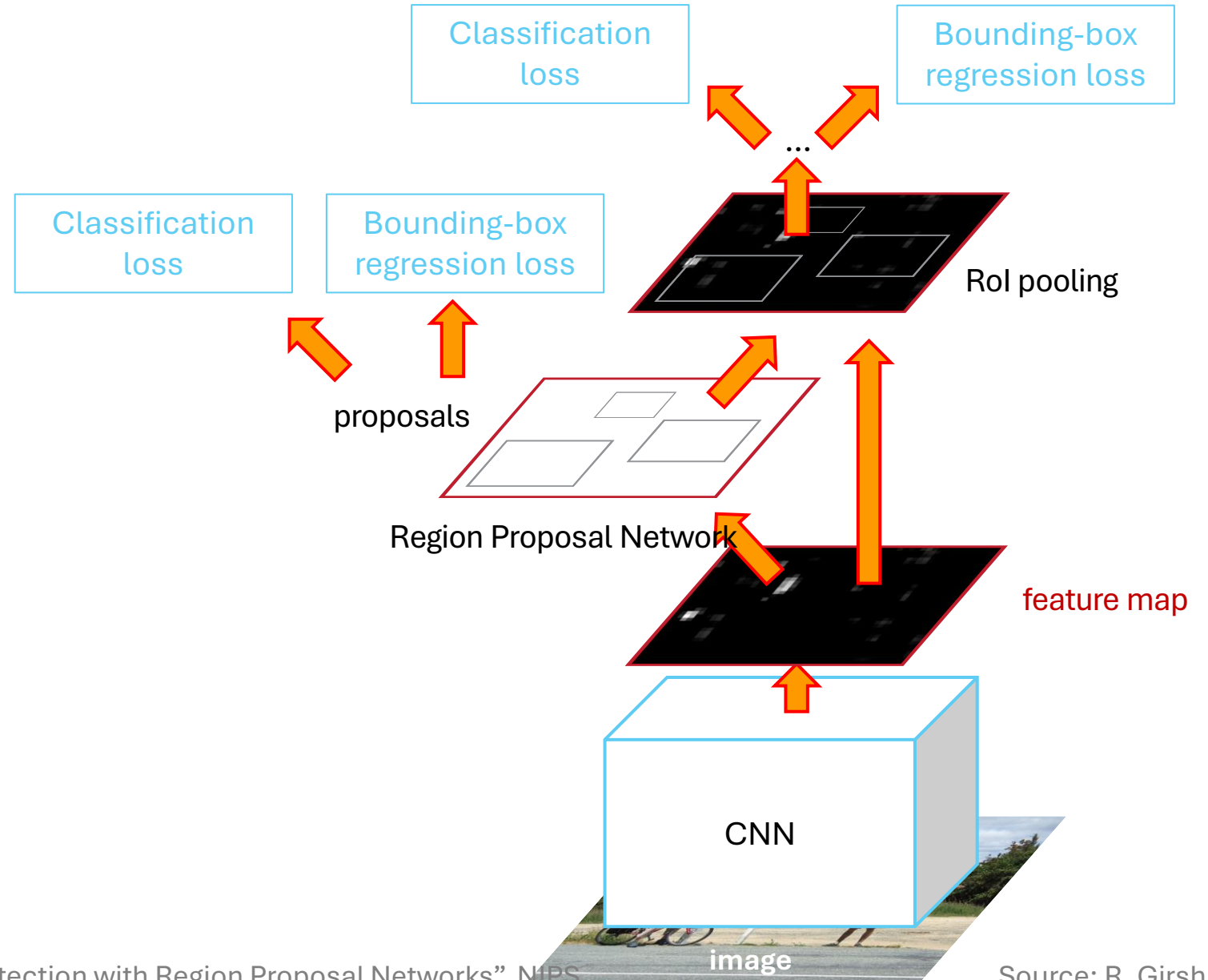
Otherwise same as Fast R-CNN:
Crop features for each proposal, classify each one



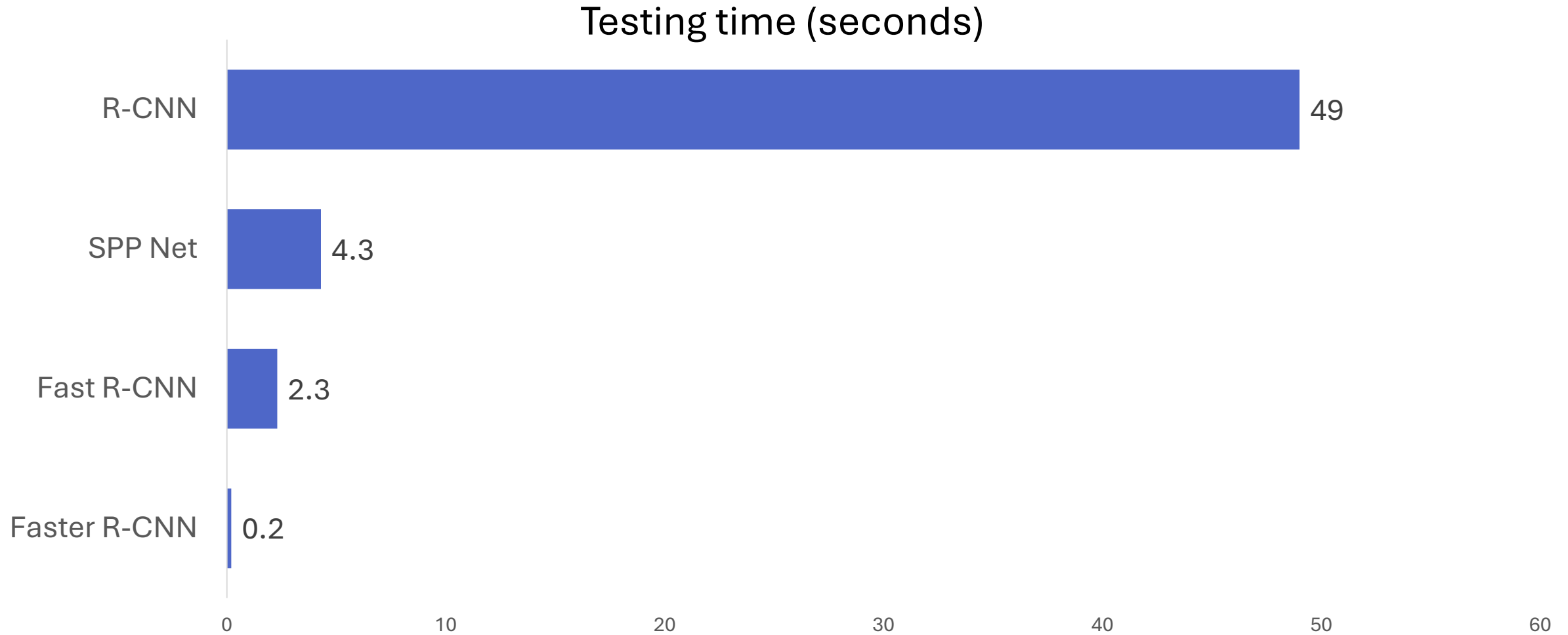
Faster R-CNN: Learnable Region Proposals

Jointly train with 4 losses:

1. **RPN classification:** anchor box is object / not an object
2. **RPN regression:** predict transform from anchor box to proposal box
3. **Object classification:** classify proposals as background /object class
4. **Object regression:** predict transform from proposal box to object box



Faster R-CNN: Test-Time Speed



Faster R-CNN: Learnable Region Proposals

Faster R-CNN is a
Two-stage object detector

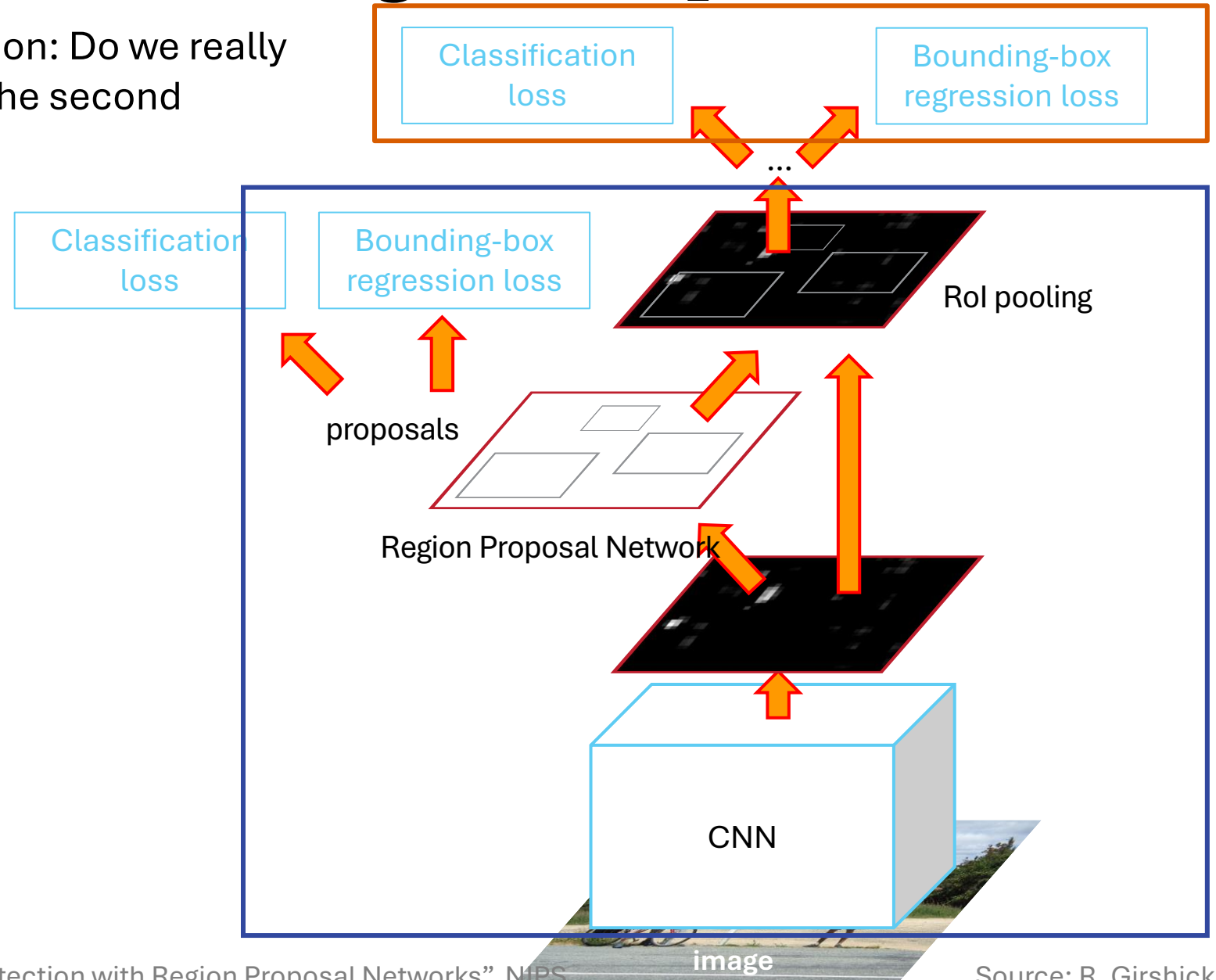
First stage: Run once per image

- Backbone network
- Region proposal network

Second stage: Run once per region

- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

Question: Do we really
need the second
stage?



Single-Stage Object Detection

Run backbone CNN to get features aligned to input image



Input Image
(e.g., $3 \times 640 \times 480$)

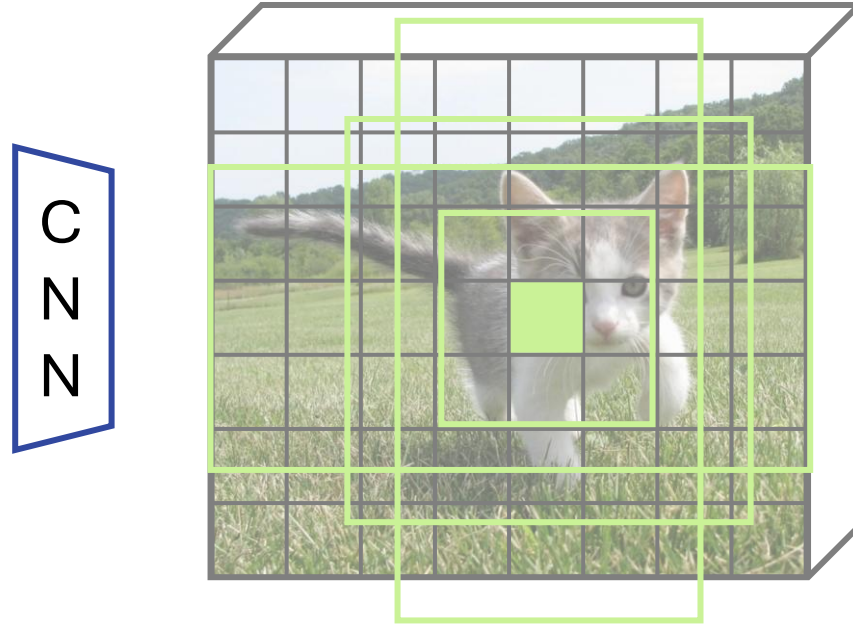
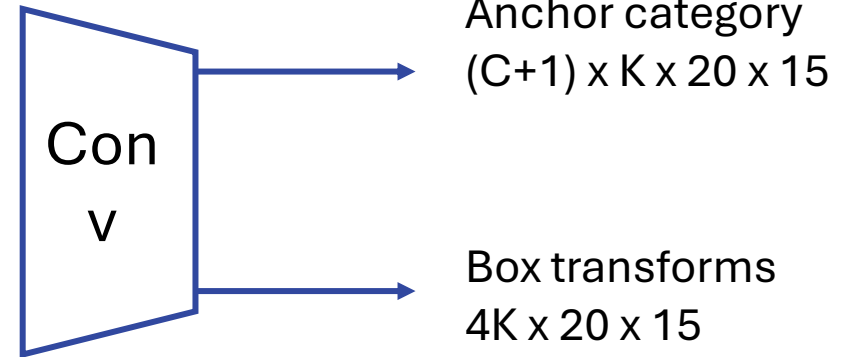


Image Features
(e.g., $512 \times 20 \times 15$)

RPN: Classify each anchor as object or non-object
Single-Stage Detector: Classify each object as one of C categories (or background)



Remember: K anchors at each position in image feature map

Single-Stage Object Detection

Run backbone CNN to get features aligned to input image



Input Image
(e.g., $3 \times 640 \times 480$)

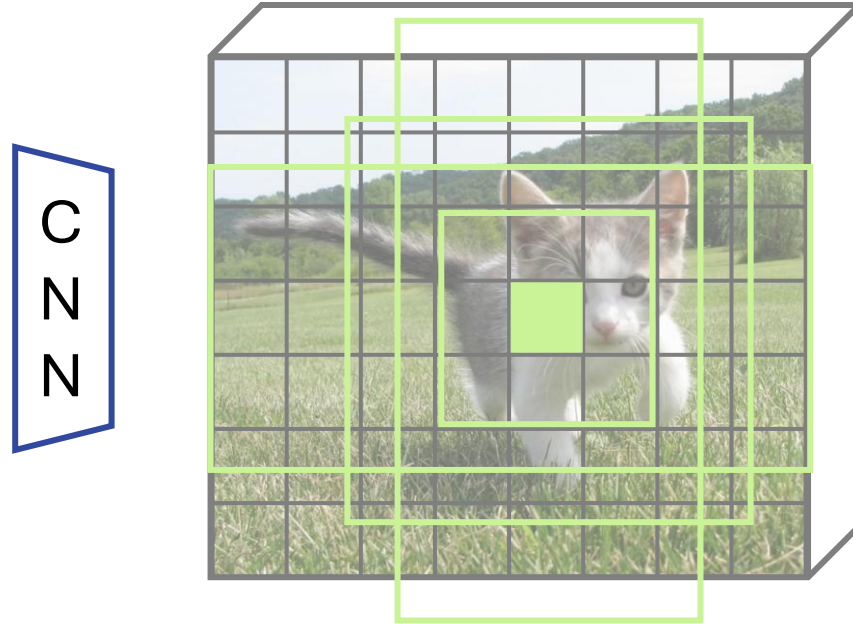
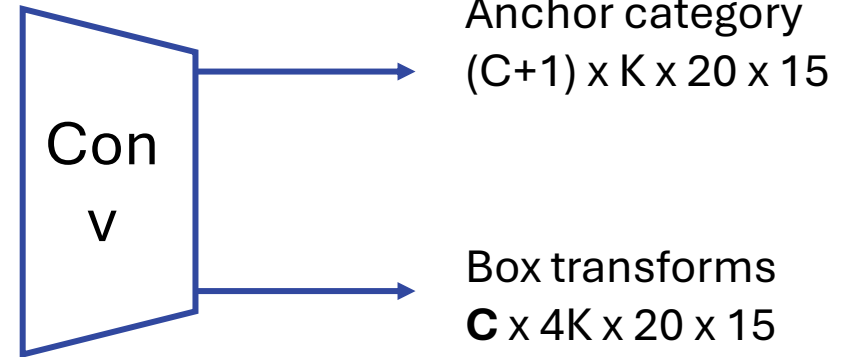


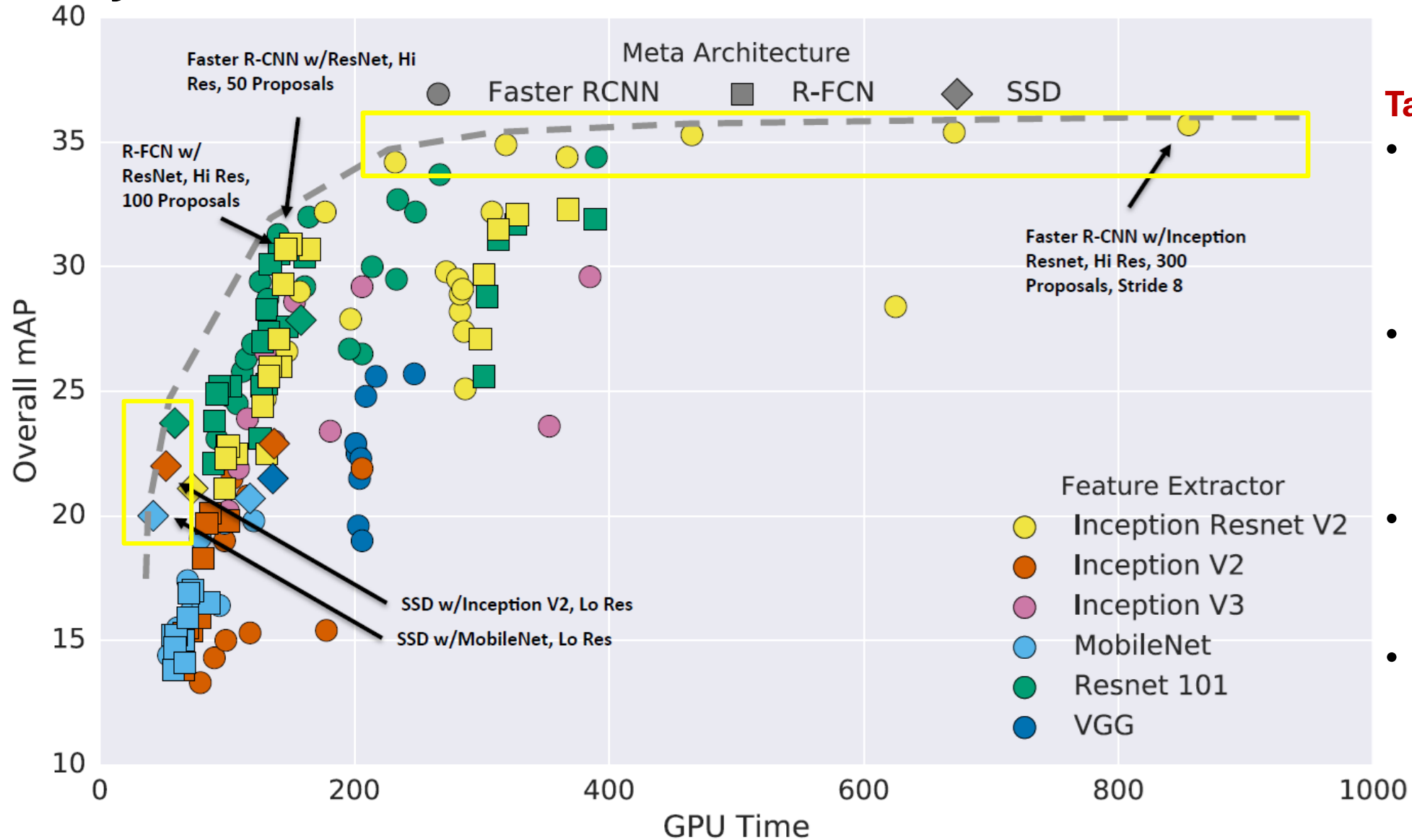
Image Features
(e.g., $512 \times 20 \times 15$)

RPN: Classify each anchor as object or non-object
Single-Stage Detector: Classify each object as one of C categories (or background)



Sometimes use **category-specific regression**: Predict different box transforms for each category

Object Detection: Lots of Variables



Takeaways:

- Two stage method (Faster R-CNN) get the best accuracy, but are slower
- Single-stage methods (SSD) are much faster, but don't perform as well
- Bigger backbones improve performance, but are slower
- Diminishing returns for slower methods

Object Detection: Open-Source Code

Object detection is hard! Don't implement it yourself

TensorFlow Detection API:

https://github.com/tensorflow/models/tree/master/research/object_detection

Faster R-CNN, SSD, RFCN, Mask R-CNN

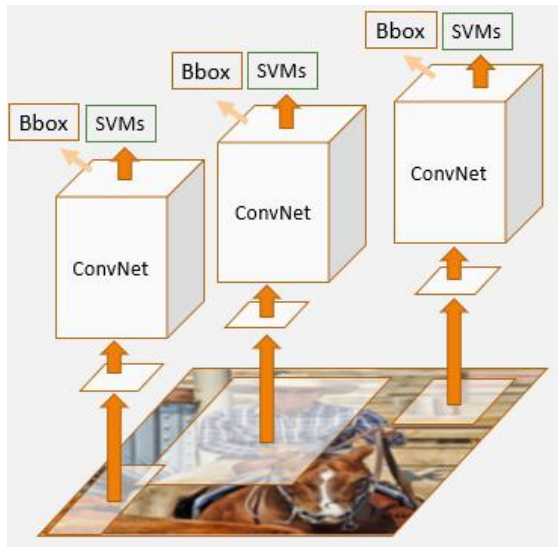
Detectron2 (PyTorch):

<https://github.com/facebookresearch/detectron2>

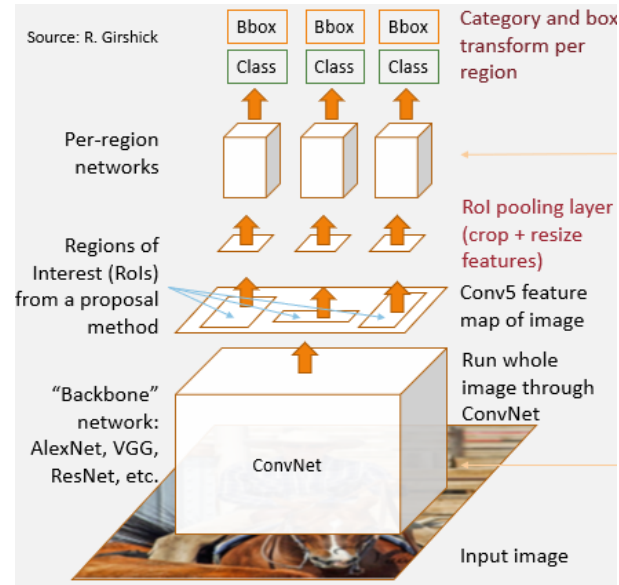
Fast / Faster / Mask R-CNN, RetinaNet

Summary

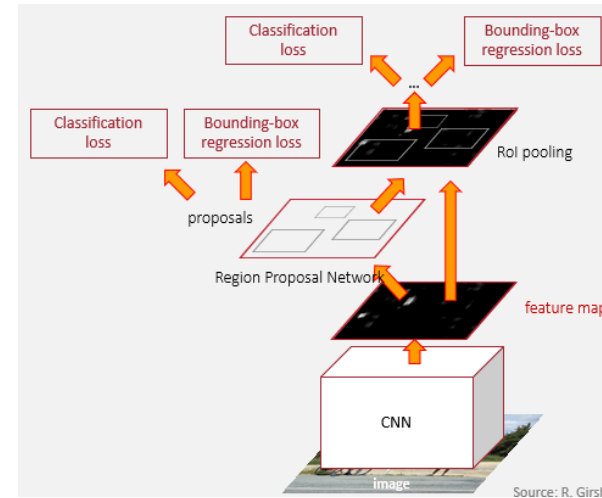
“Slow” R-CNN: Run CNN independently for each region



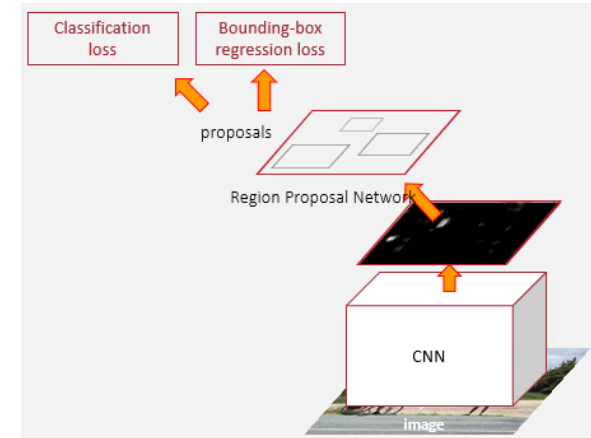
Fast R-CNN: Apply differentiable cropping to shared image features



Faster R-CNN: Compute proposals with CNN



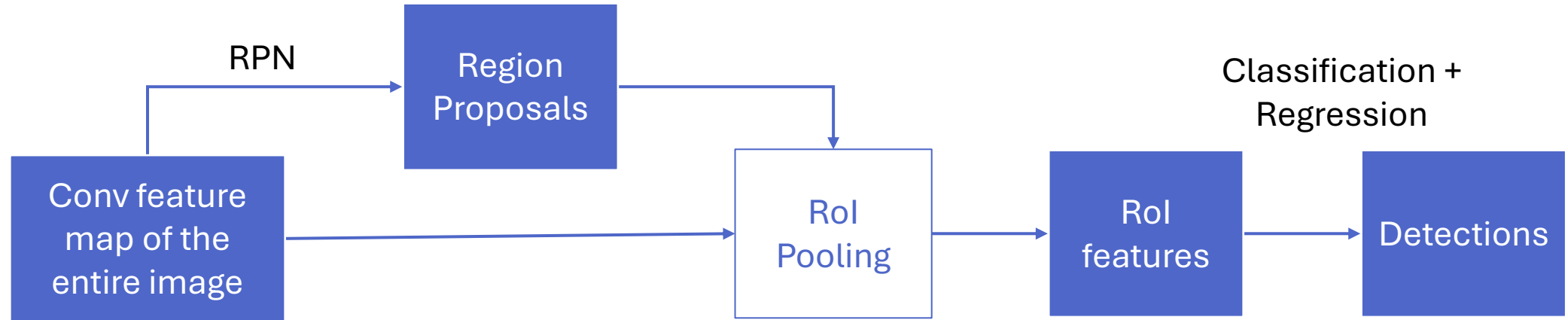
Single-Stage: Fully convolutional detector



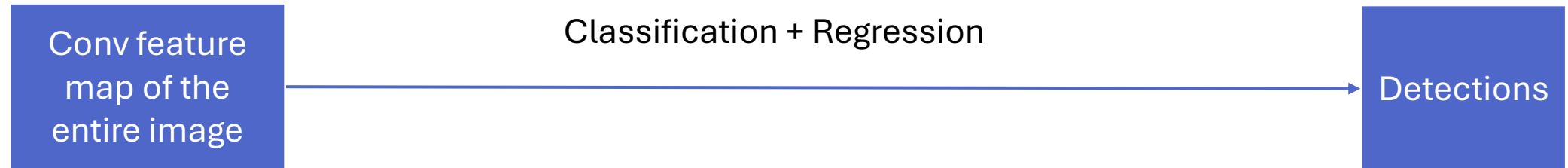
Single-Stage Multi-Resolution Detectors

Streamlined detection architectures

- The Faster R-CNN pipeline separates proposal generation and region classification



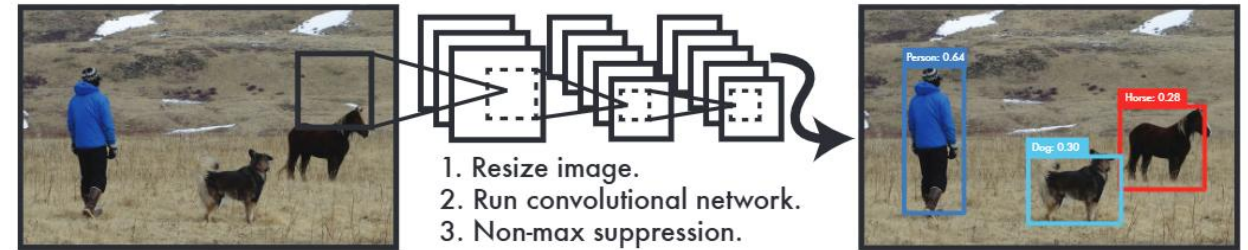
- Is it possible to do detection in one shot?



YOLO

- Divides the image into an $S \times S$ grid. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object.
- Each grid cell predicts B bounding boxes (BB) and confidences scores for those boxes

$$C_i = P_r(\text{Object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}$$
- Reflects how confident the models is that the box contains an object and also how accurate it thinks the box is that it predicts.
- Each BB consists of 5 predictions:
 - (x, y) : center of the box relative to the bounds of the grid cell.
 - (w, h) : width and height relative to the whole image
- Confidence: IOU between the predicted box and any ground truth box.

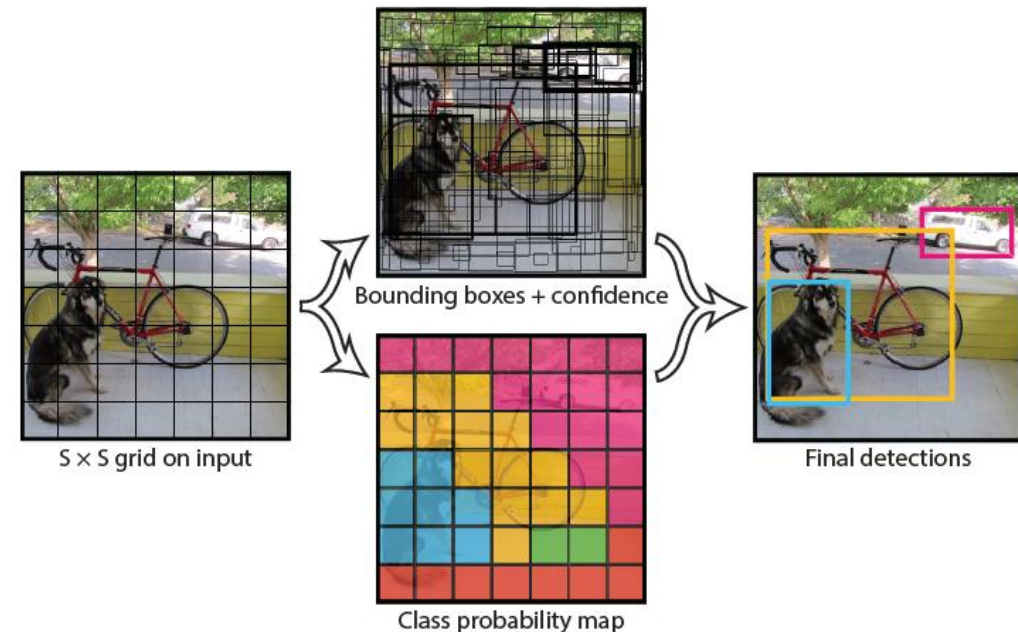


Each grid cell also predicts C conditional class probabilities, $\text{Pr}(\text{Class}_i | \text{Object})$

$$\text{Pr}(\text{Class}_i | \text{Object}) \times P_r(\text{Object}) \times \text{IoU}_{\text{pred}}^{\text{truth}} = \text{Pr}(\text{Class}_i) \times \text{IoU}_{\text{pred}}^{\text{truth}}$$

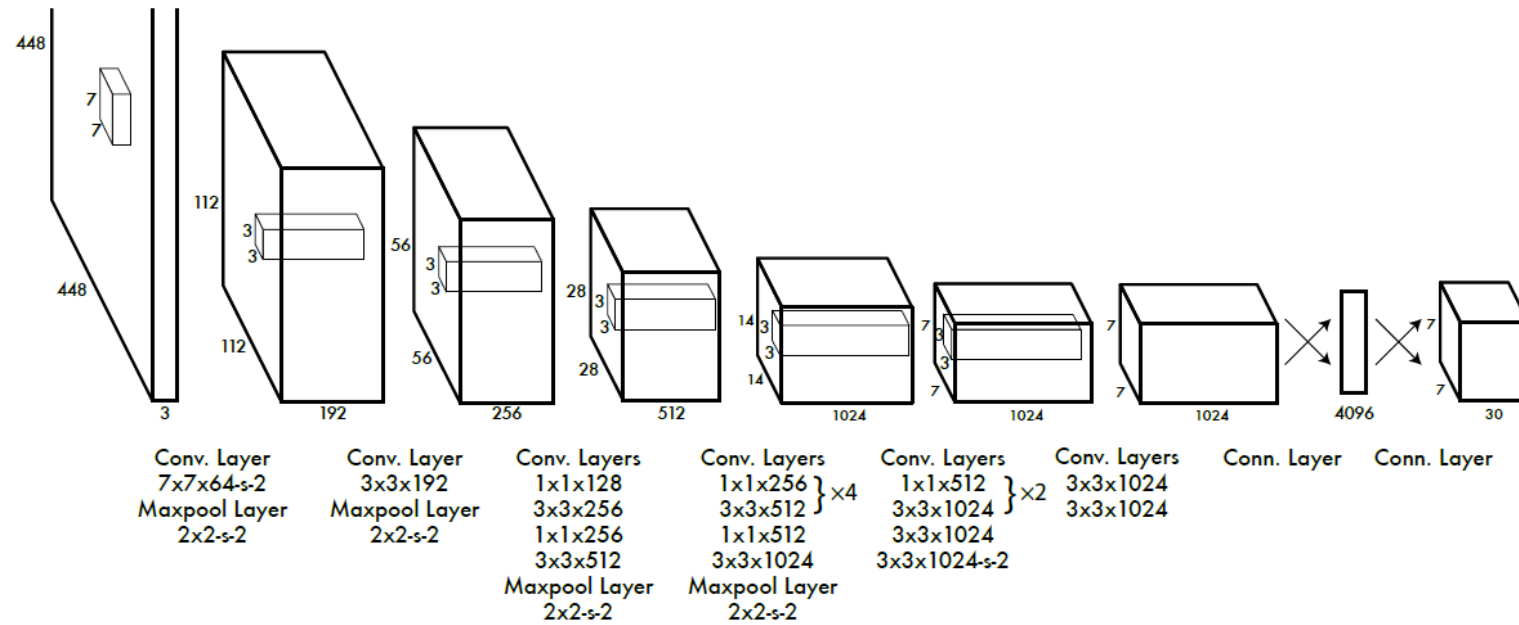
Predicts one set of class probabilities per grid cell, regardless of the number of boxes B .

Output tensor shape: $S \times S \times (B \times 5 + C)$



YOLO

1. Take conv feature maps at 7x7 resolution
2. Add two FC layers to predict, at each location, a score for each class and 2 bounding box confidences
 - For PASCAL, output is $7 \times 7 \times 30$ ($30 = 20 + 2 * (4 + 1)$)



Ground Truth for YOLO

In the YOLO algorithm, if you use 7×7 cells and there are 20 classes, for each training image, what will the ground truth tensor look like?

Input image is divided into an $S \times S$ **grid**. Here: $S = 7$, so the grid has $7 \times 7 = 49$ cells. Each grid cell is responsible for predicting objects whose **center falls inside that cell**.

For each cell, YOLO predicts:

Bounding box coordinates (x, y, w, h) , (x, y) relative to the cell and (w, h) relative to the image.

Confidence score(s): probability the box contains an object $\times$ IoU with ground truth.

Class probabilities: distribution over all C classes. Here: $C = 20$.

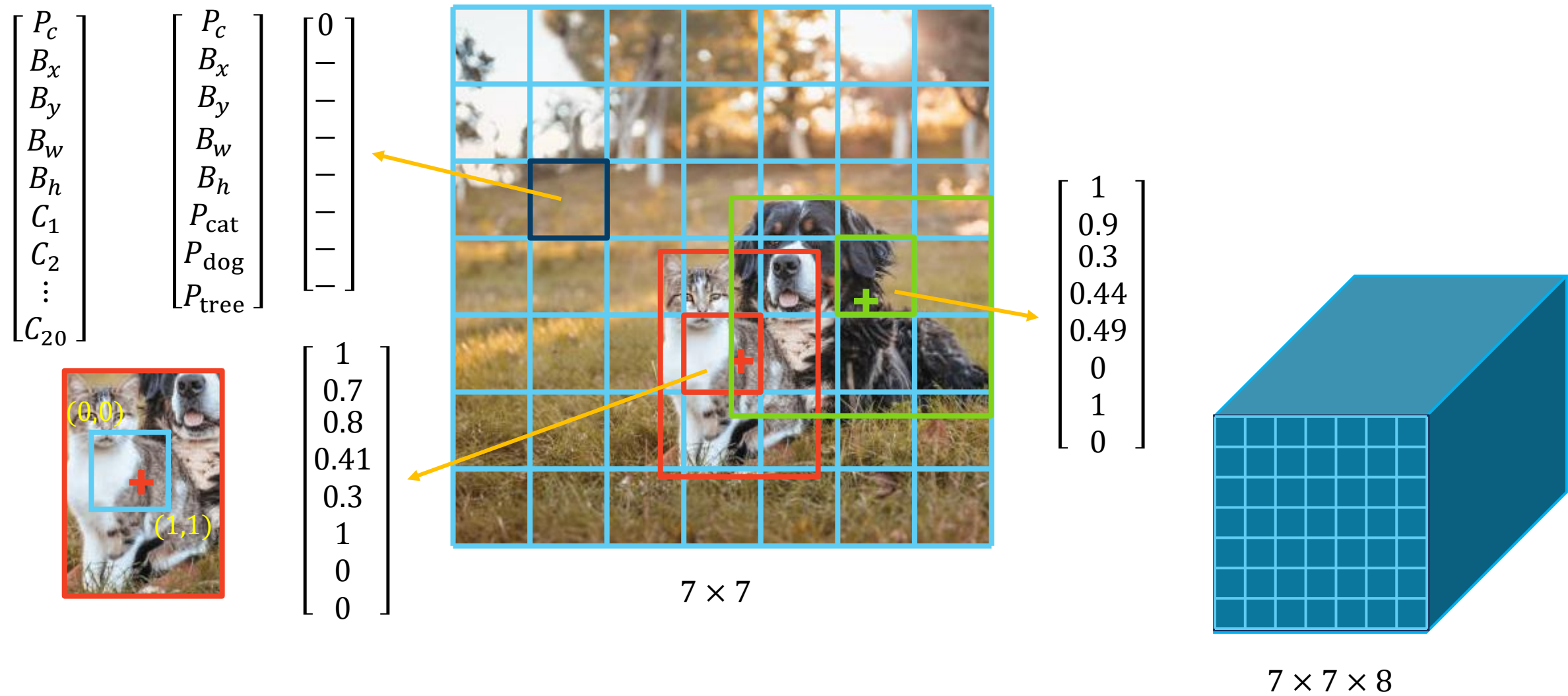
In YOLOv1, each cell predicts B bounding boxes (they used $B = 2$ in the paper).

i.e., $5B + C = 5 \times 2 + 20 = 30$ numbers per cell.

$[x_1, y_1, w_1, h_1, c_1, x_2, y_2, w_2, h_2, c_2, p_1, p_2, \dots, p_{20}]$

Ground truth tensor: $7 \times 7 \times 30$

How YOLO Works



YOLO

Objective function:

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

Regression

Object/no object confidence

Class prediction

YOLO

Objective function:

$$\begin{aligned}
 & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
 & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2 \\
 & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} \left(p_i(c) - \hat{p}_i(c) \right)^2
 \end{aligned}$$

Cell i contains object, predictor j is responsible for it

Small deviations matter less for larger boxes than for smaller boxes

Confidence for object

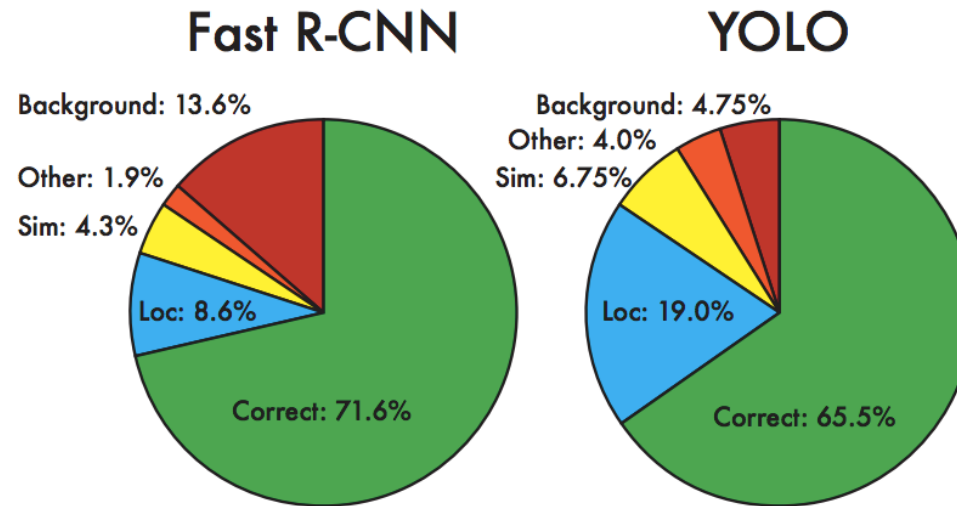
Confidence for no object

Down-weight loss from boxes that don't contain objects ($\lambda_{\text{noobj}} = 0.5$)

Class probability

YOLO: Results

- Each grid cell predicts only two boxes and can only have one class: this limits the number of nearby objects that can be predicted, poor at handling aspect ratios and scales.
- Grid: hard to detect small object, limits the total no. of objects detectable, due to less precise BB regression, results in localization errors.
- Localization accuracy suffers compared to Fast(er) R-CNN due to coarser features, errors on small boxes
- 7x speedup over Faster R-CNN (45-155 FPS vs. 7-18 FPS)

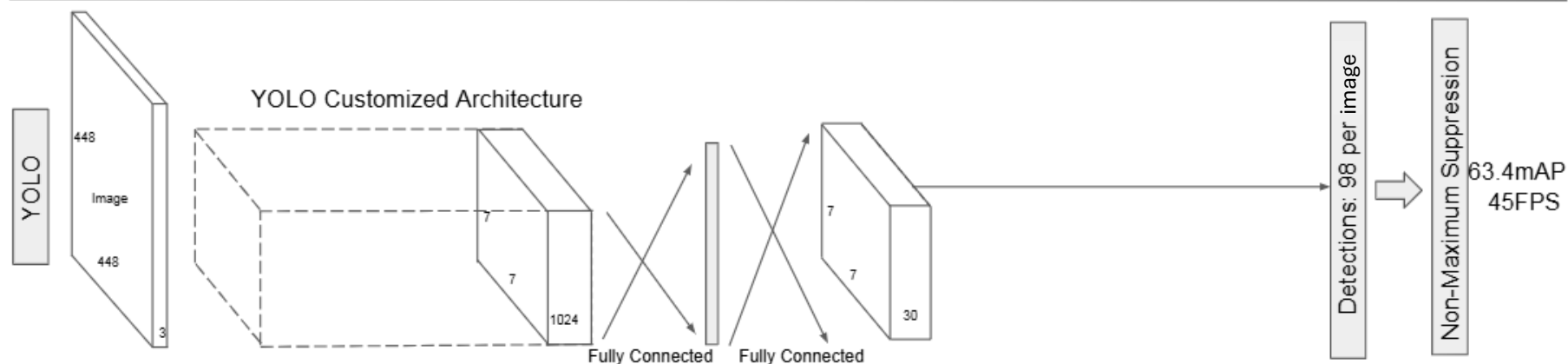
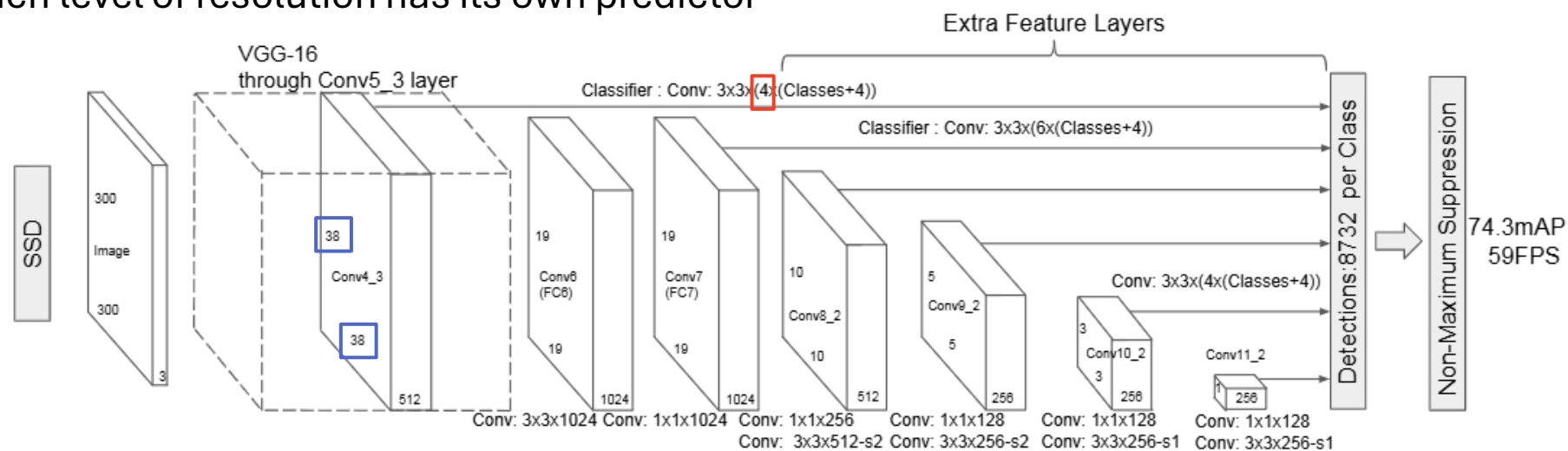


Performance on PASCAL 2007

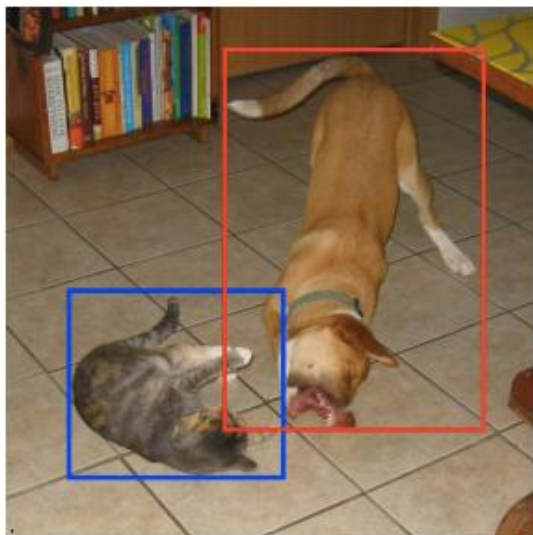
Multi-Resolution Prediction: SSD

- Predict boxes of different size from different conv maps
- Each level of resolution has its own predictor

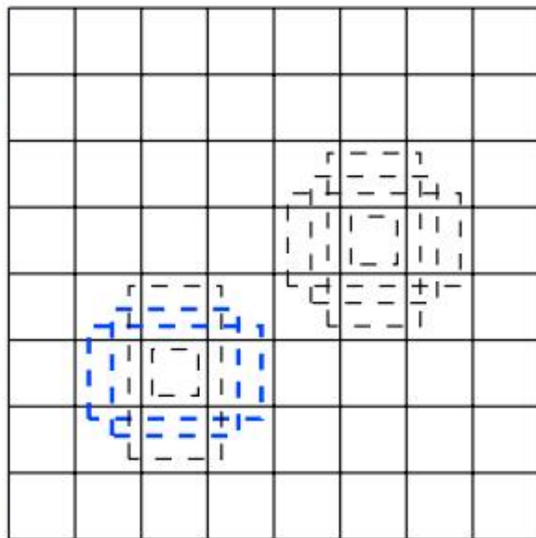
$$8732 = 4 \times 38 \times 38 + 6 \times 19 \times 19 + 6 \times 10 \times 10 + 6 \times 5 \times 5 + 4 \times 3 \times 3 + 4 \times 1 \times 1$$



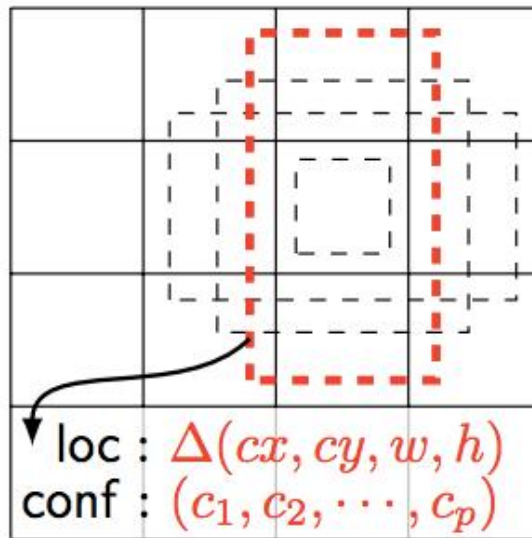
Multi-Resolution Prediction: SSD



(a) Image with GT boxes



(b) 8×8 feature map



(c) 4×4 feature map

Set of default bounding boxes with each feature map cell, for multiple feature maps.

At each feature map cell, SSD predicts the offsets relative to the default box shapes in the cell.

It also predicts the per-class scores that indicate the presence of a class instance in each of those boxes.

c = no. of class scores

k = no. of boxes

$(c + 4)kmn$ outputs for an $m \times n$ feature map.

$x_{ij}^p = \{1, 0\}$ = indicator for whether the i th default box matches the j th GT box for category p .

$$L(x, c, l, g) = \frac{1}{N} (L_{\text{conf}}(x, c) + \alpha L_{\text{loc}}(x, l, g))$$

no. of matched default boxes

GT box parameters

predicted box parameters

Outline

- Task definition and evaluation
- Two-stage detectors
 - R-CNN
 - Fast R-CNN
 - Faster R-CNN
 - Mask R-CNN
- Single-stage and multi-resolution detectors
- Other detectors
 - RetinaNet
 - CornerNet
 - CenterNet
 - DETR

CornerNet: Detecting Object as Paired Keypoints

Detects a pair of corners of a bounding box and groups them to form the final detected bounding box.

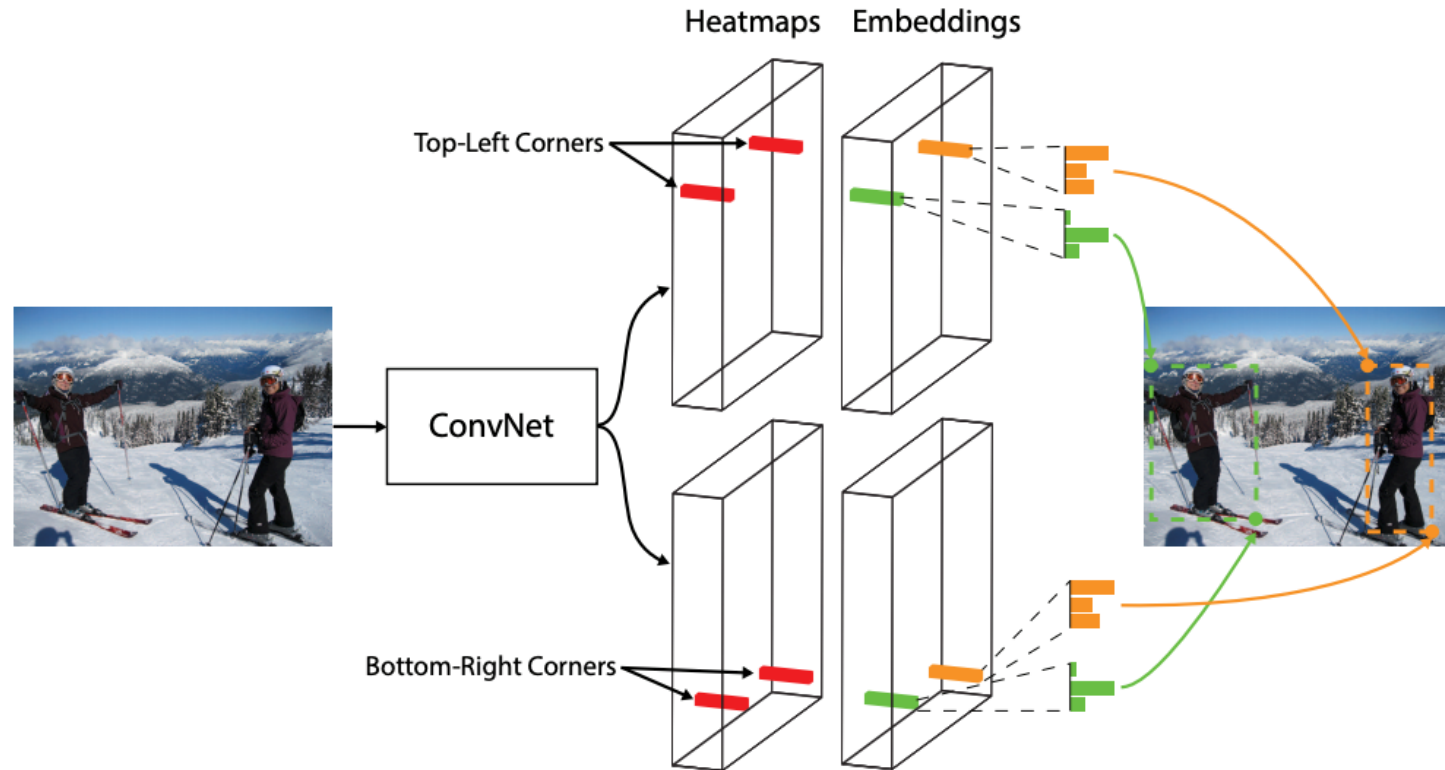
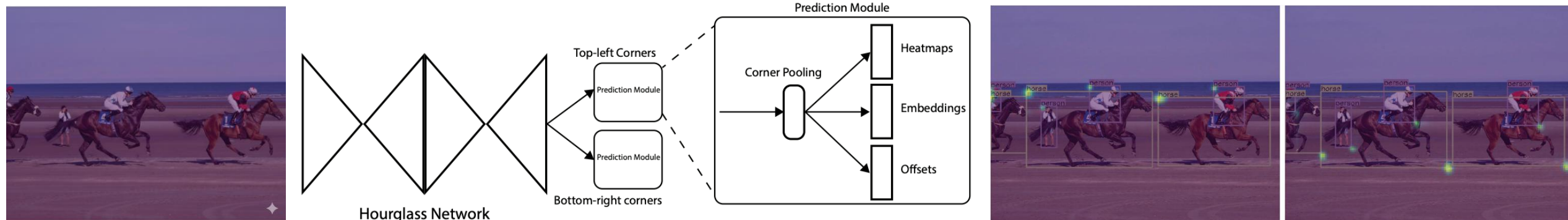


Fig. 1. We detect an object as a pair of bounding box corners grouped together. A convolutional network outputs a heatmap for all top-left corners, a heatmap for all bottom-right corners, and an embedding vector for each detected corner. The network is trained to predict similar embeddings for corners that belong to the same object.

CornerNet: Detecting Object as Paired Keypoints

Heat maps: top-left corners, bottom-right corners



Corner Pooling; As there is no visual evidence for the presence of corners.

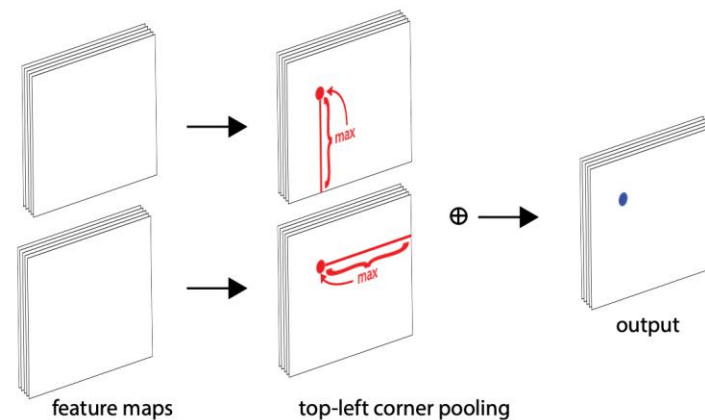


Fig. 2. Often there is no local evidence to determine the location of a bounding box corner. We address this issue by proposing a new type of pooling layer.

Detection Transformer (DETR)

